

AVL TREES





INTRODUCTION

- Invented in 1962 by Russian mathematicians Adelson-Velski and Landis
- An AVL tree (originally called admissible) tree is one in which the height of the left and right sub tree of every node differ by at most 1.
- It is a binary search tree too.





INTRODUCTION

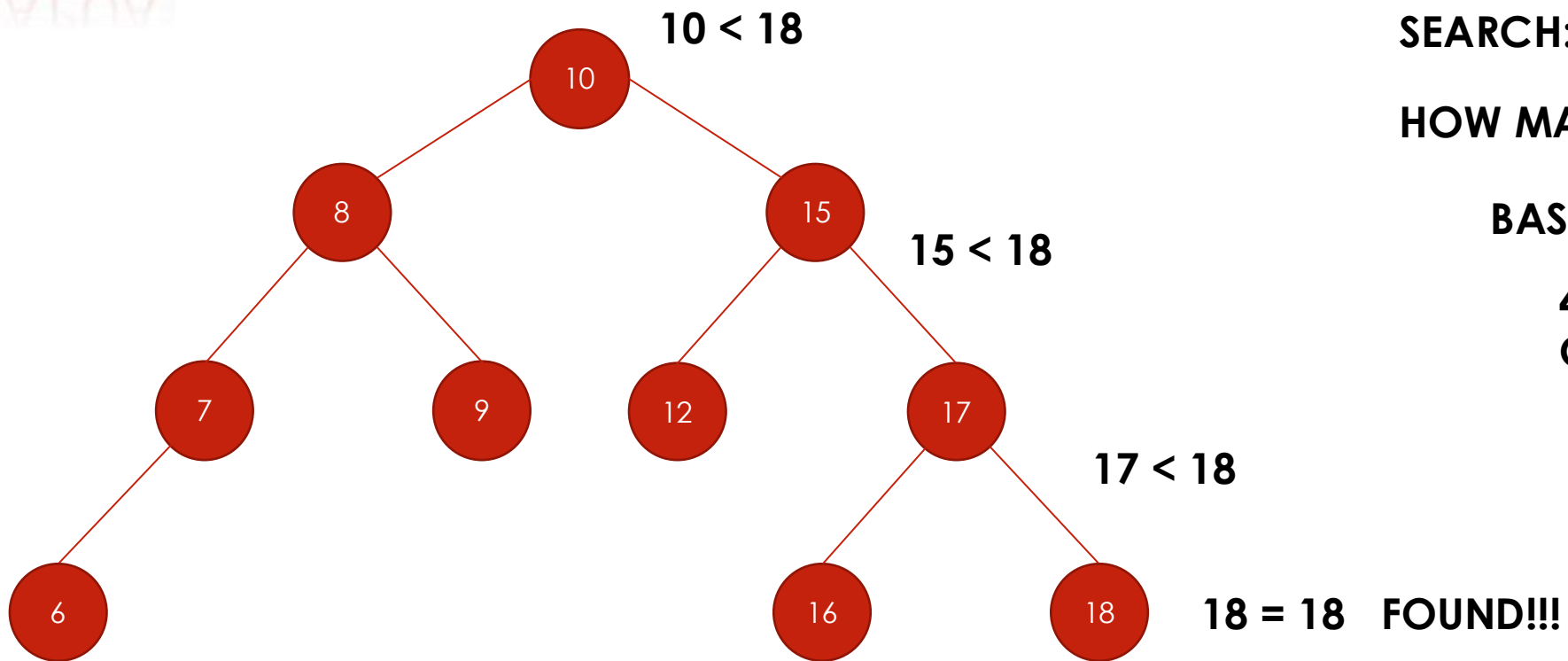
- Balance factor of a node is defined as follows:

$\text{height}(\text{left sub tree}) - \text{height}(\text{right sub tree})$

- i.e., for an AVL tree, all balance factors should be **+1, 0 or -1**



BINARY SEARCH TREE

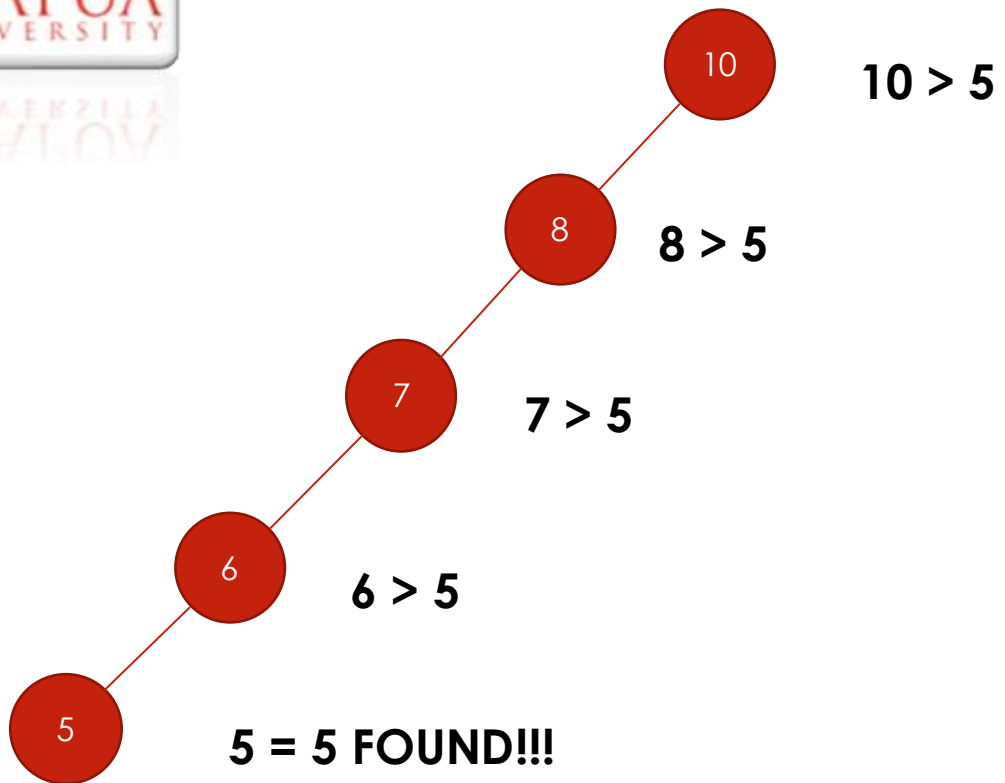


SEARCH: 18

HOW MANY STEPS?

BASED ON HEIGHT

4 STEPS
 $O(H)$



BINARY SEARCH TREE

SEARCH: 5

HOW MANY STEPS?

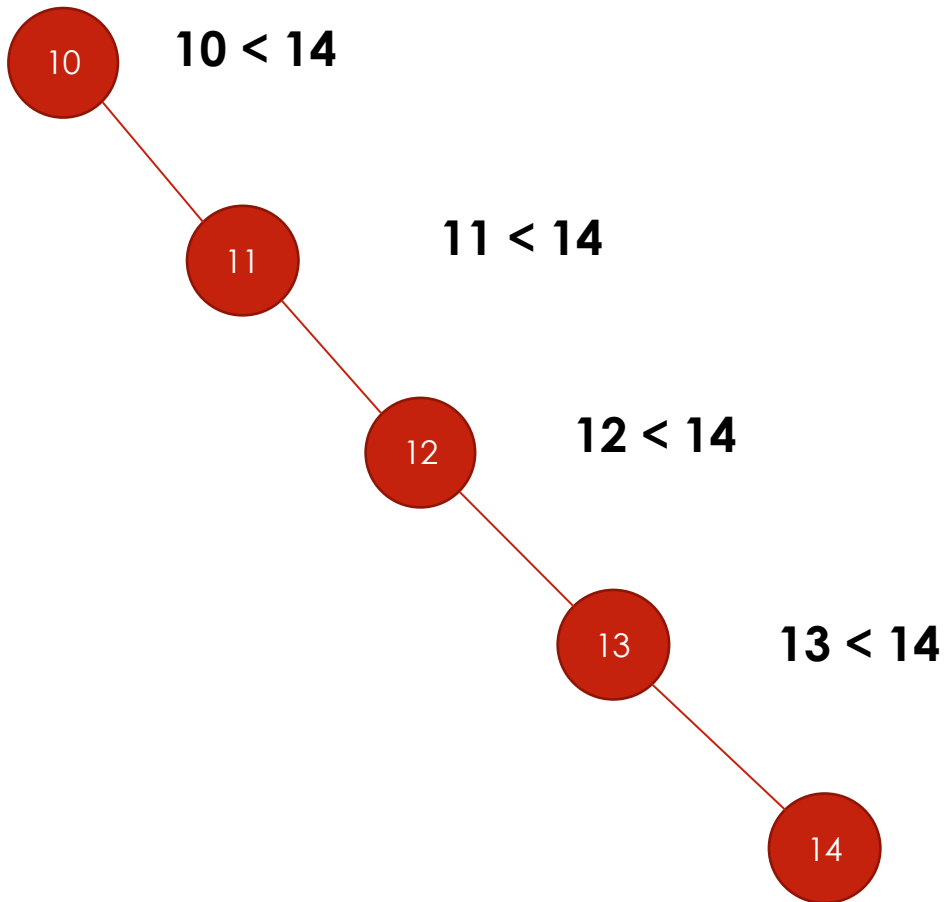
BASED ON HEIGHT

5 STEPS

$H = N$

$O(N)$

BINARY SEARCH TREE



SEARCH: 14

HOW MANY STEPS?

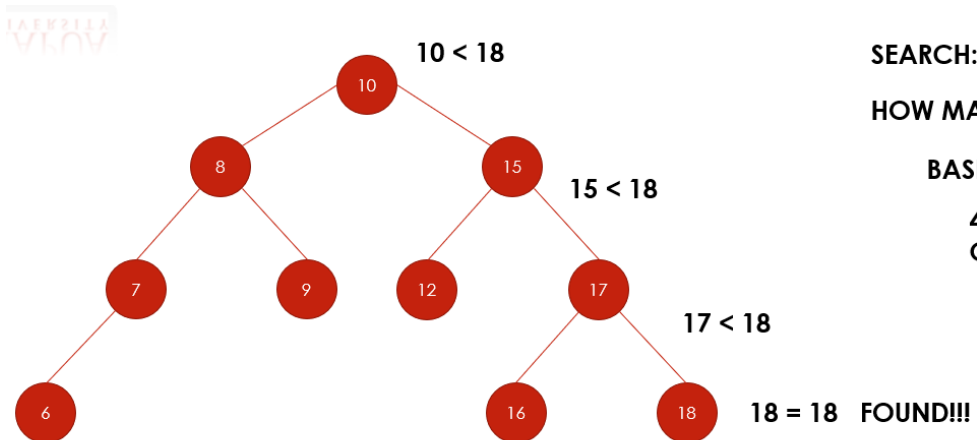
BASED ON HEIGHT

5 STEPS

$H = N$

$O(N)$

WHICH IS FASTER???

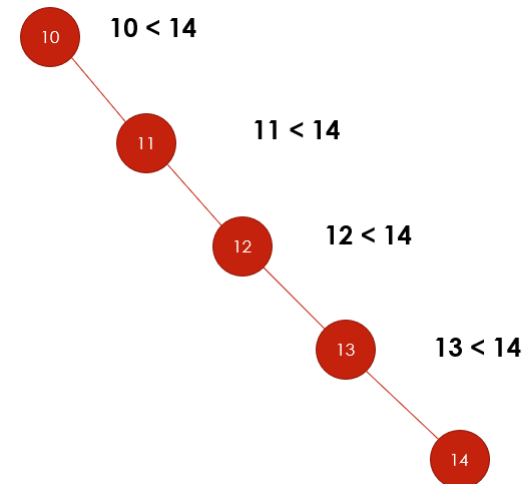


SEARCH: 18

HOW MANY STEPS?

BASED ON HEIGHT

4 STEPS
 $O(H)$



SEARCH: 14

HOW MANY STEPS?

BASED ON HEIGHT

5 STEPS
 $H = N$
 $O(N)$

FASTER

BASIS OF AVL TREE





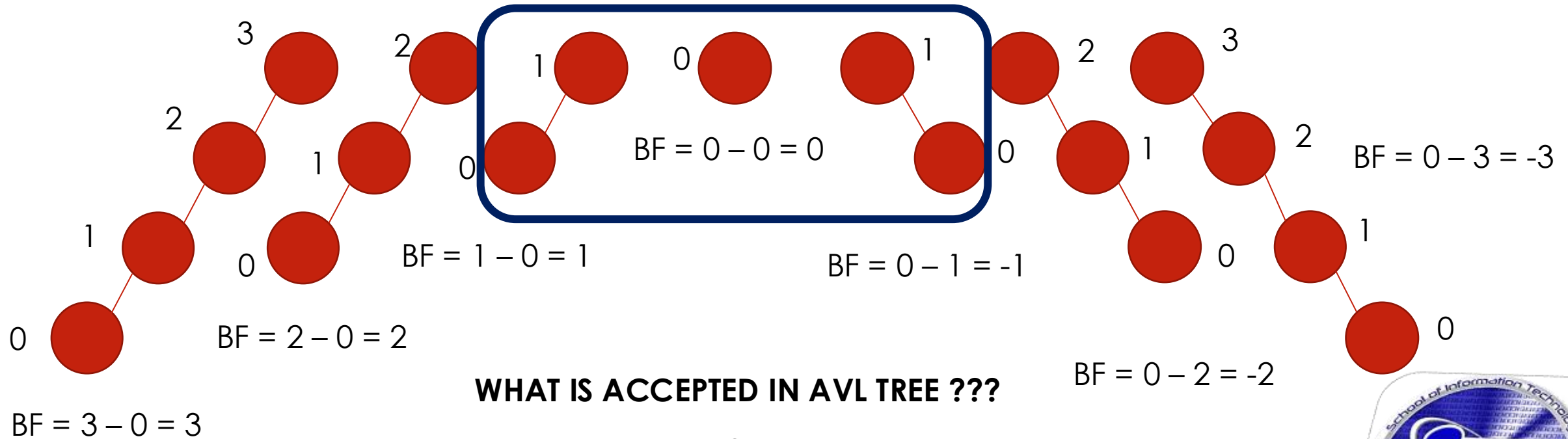
WHY AVL TREE???

- **To minimize worst case scenario**
 - From $O(N)$ to $O(H)$ or $O(\log N)$
 - Where in
 - N = number of elements
 - H = Height of the tree



BALANCE FACTOR

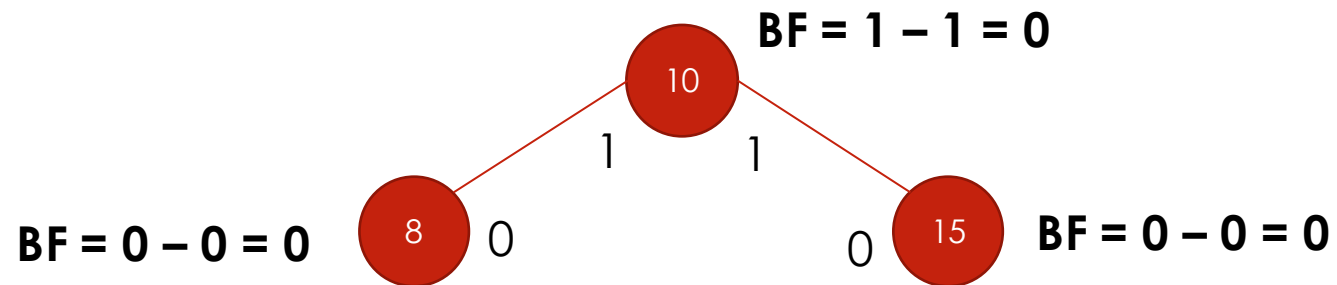
- BALANCE FACTOR = HEIGHT OF LEFT SUBTREE – HEIGHT OF RIGHT SUBTREE**



WHAT IS ACCEPTED IN AVL TREE ???

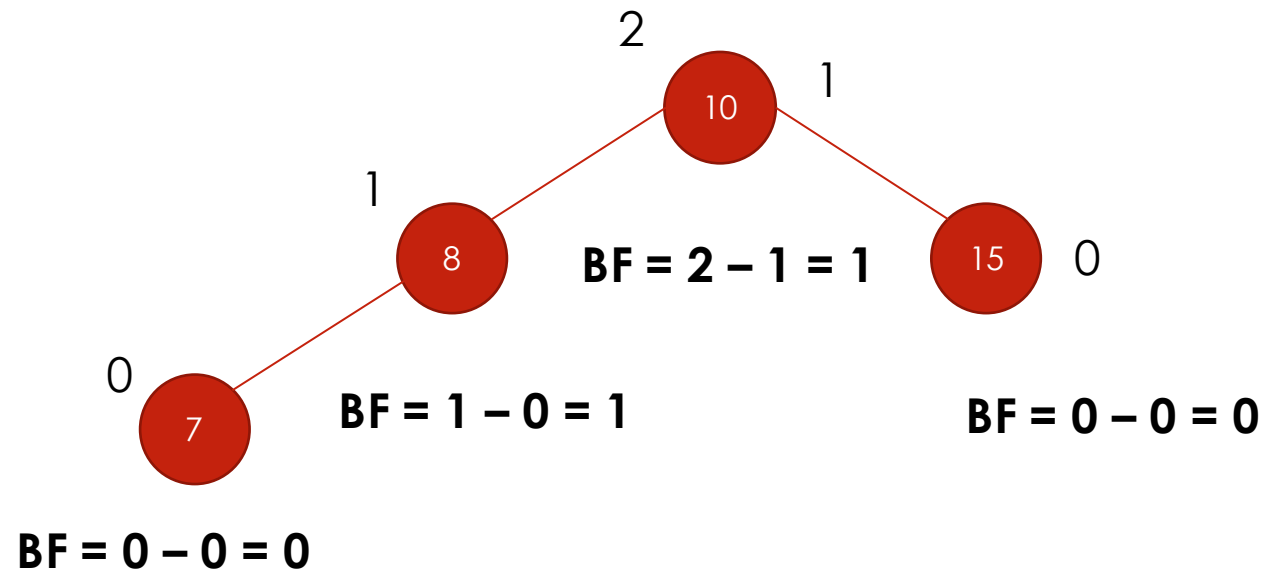
BF = -1, 0, 1

EXAMPLE



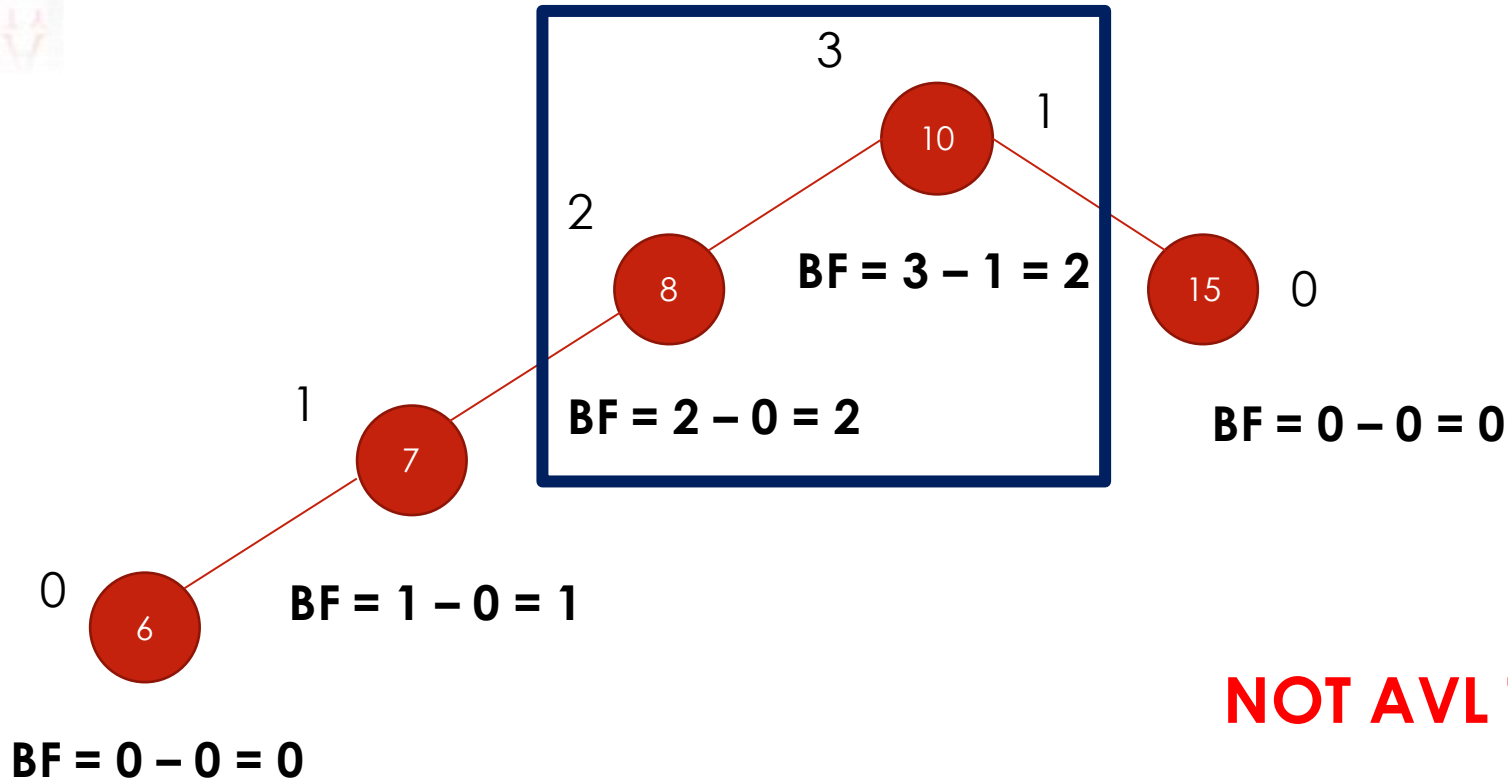
AVL TREE !!!

EXAMPLE



AVL TREE !!!

EXAMPLE



NOT AVL TREE !!!

WHAT TO DO???



ROTATIONS

- **Rotations are to rearrange nodes to maintain AVLness.**
 - i.e., the height of any node should be either -1, 0 or 1
- **They are needed on insert and delete operations.**



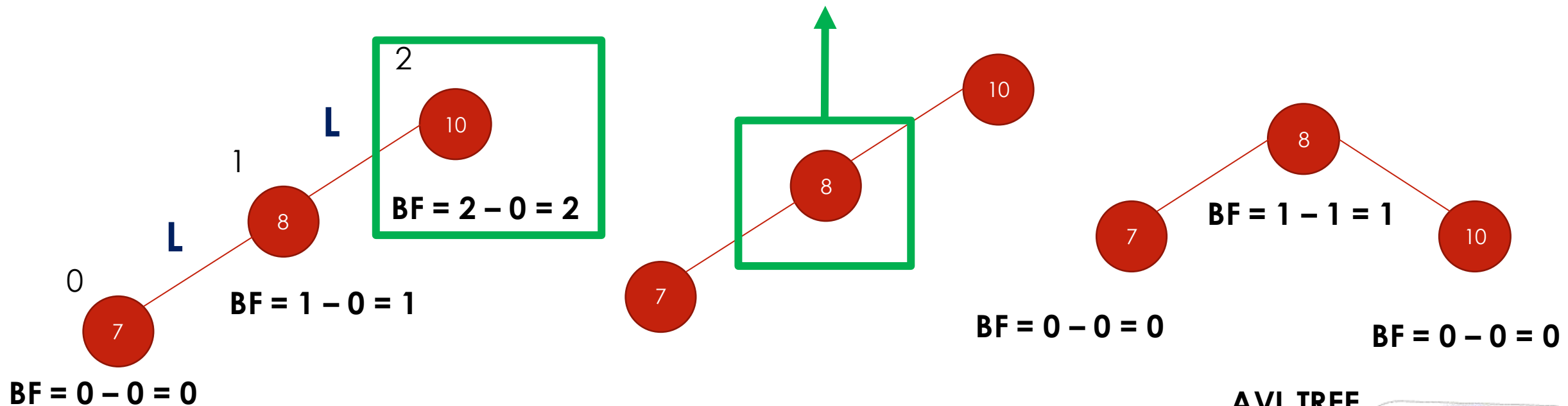


CLASSIFICATION OF ROTATIONS

- **There are four of them**
 - Single Left (LL)
 - Single Right (RR)
 - Double Left-Right (LR)
 - Double Right-Left (RL)



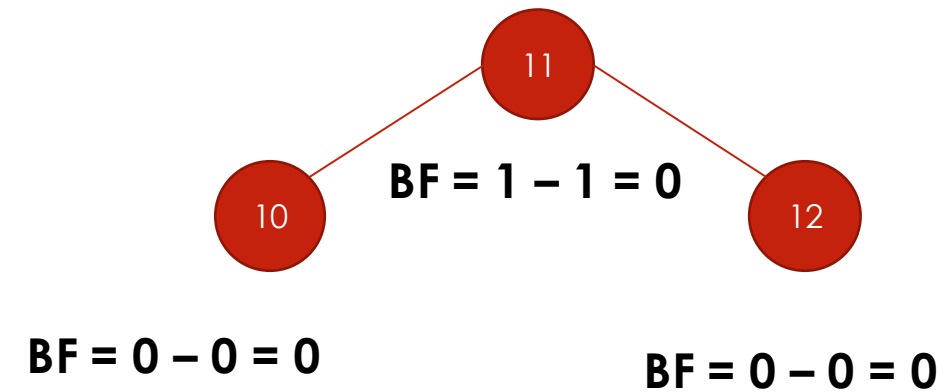
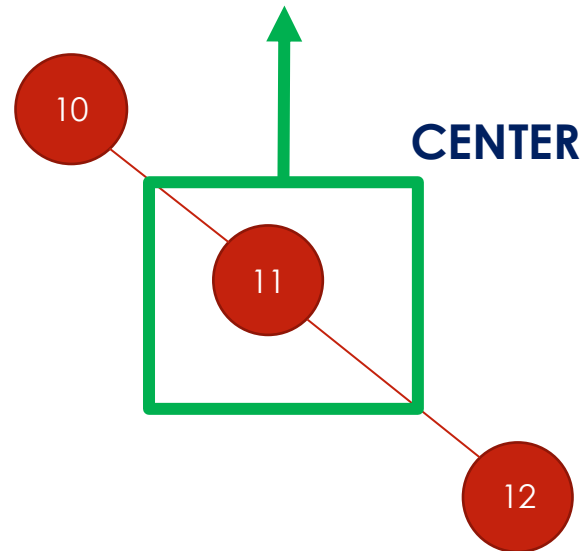
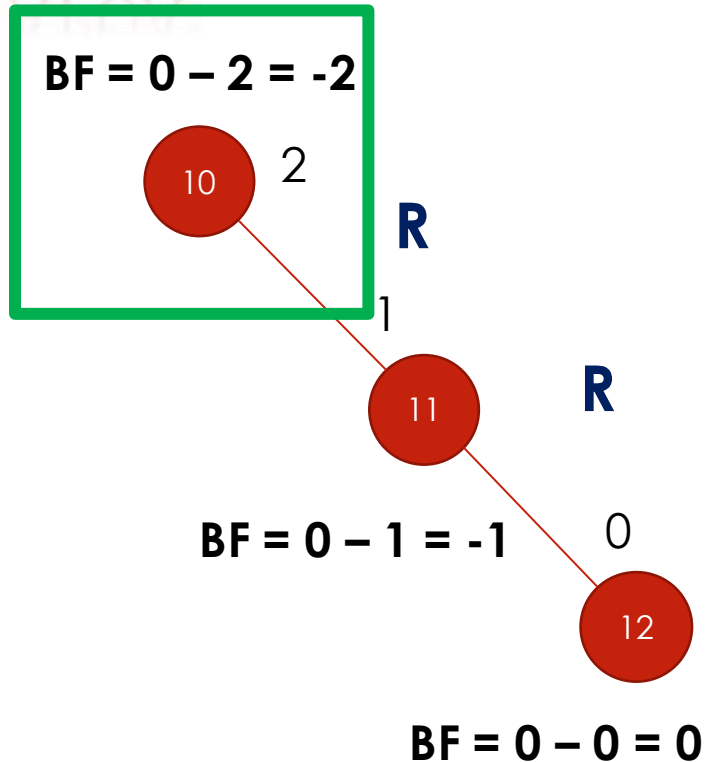
SINGLE LEFT (LL)



AVL TREE

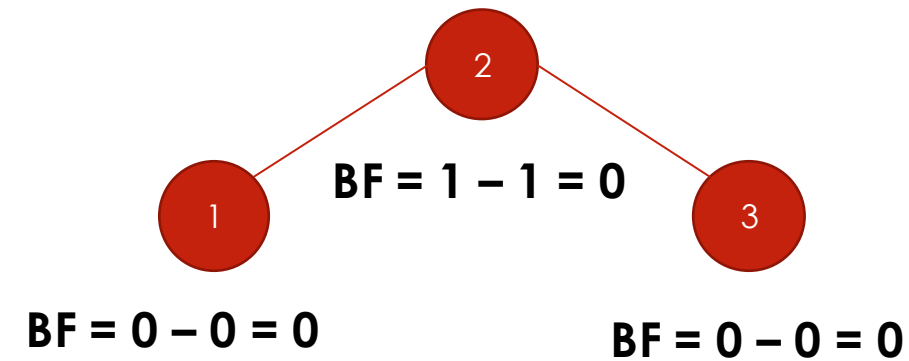
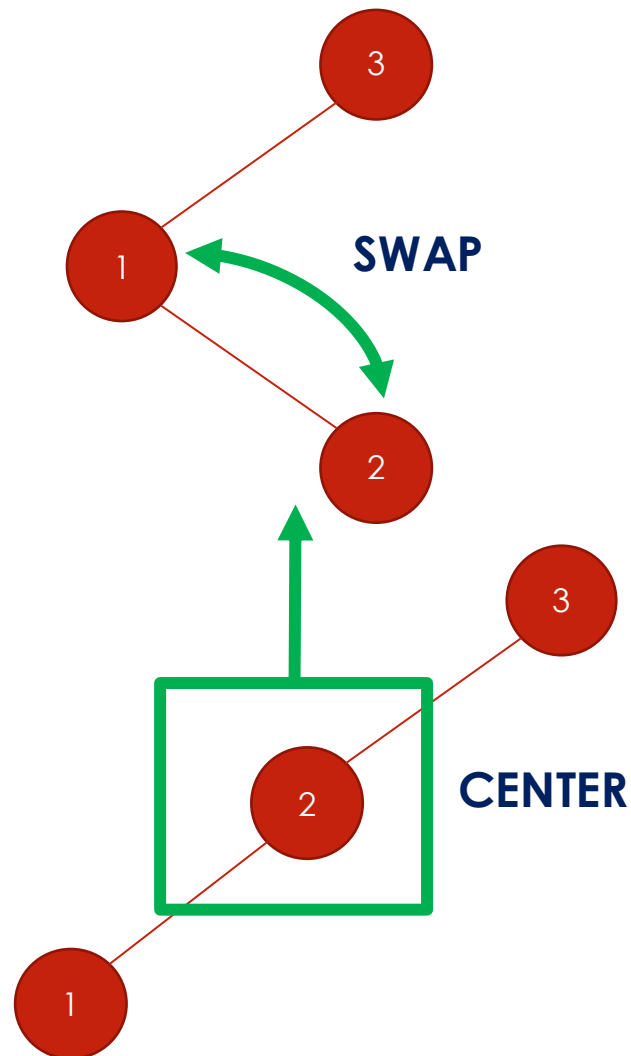
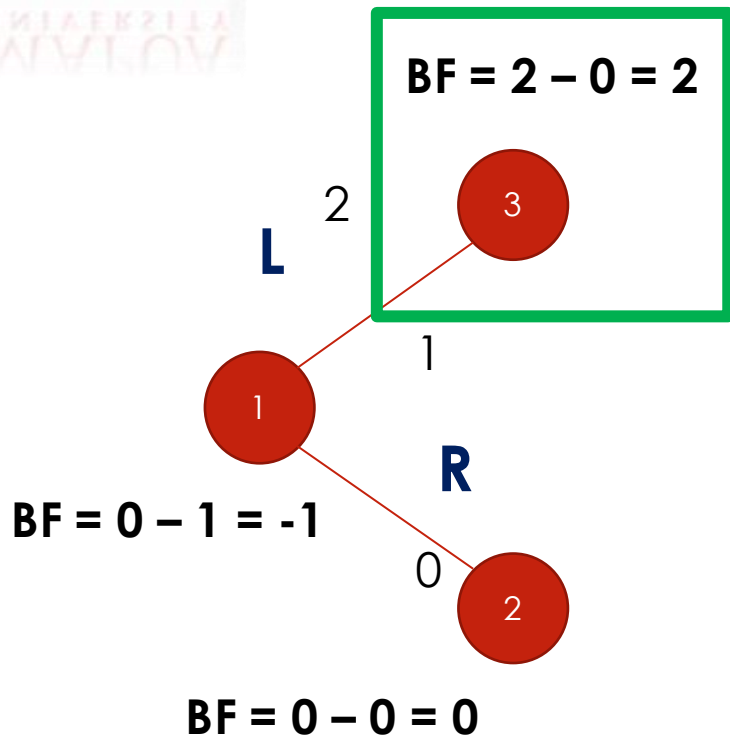


SINGLE RIGHT (RR)

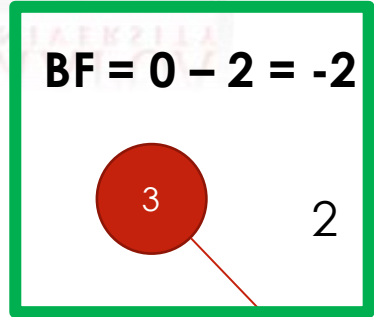


AVL TREE

LEFT-RIGHT (LR)



AVL TREE



R



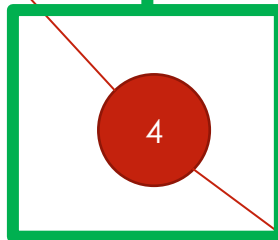
$BF = 1 - 0 = 1$

L



$BF = 0 - 0 = 0$

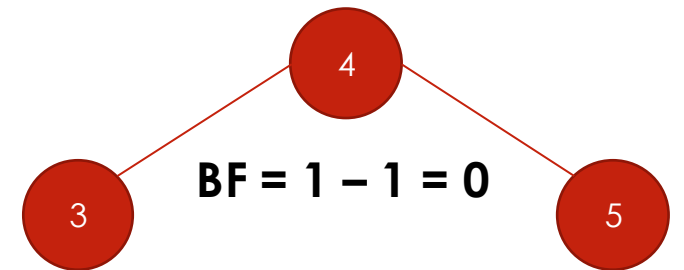
SWAP



CENTER



RIGHT-LEFT (RL)



$BF = 0 - 0 = 0$

$BF = 1 - 1 = 0$

$BF = 0 - 0 = 0$

AVL TREE

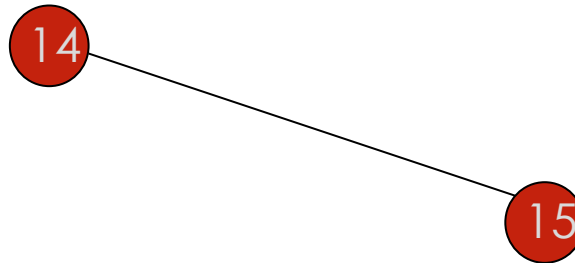


AVL TREE INSERTION EXAMPLE

AVL TREE INSERTION –EXAMPLE 1

Single rotations: **insert** 14, 15, 16, 13, 12, 11, 10

- First insert 14 and 15:

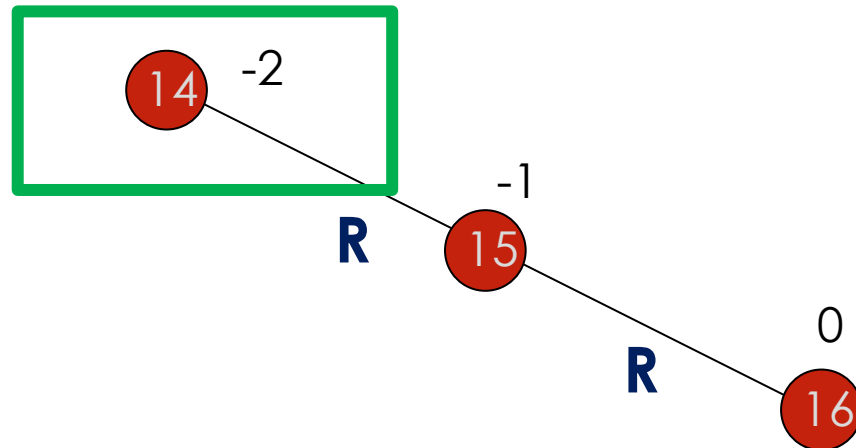


- Now insert 16.

AVL TREE INSERTION –EXAMPLE 1 (CONTD..)

Single rotations:

- Inserting 16 causes AVL violation:

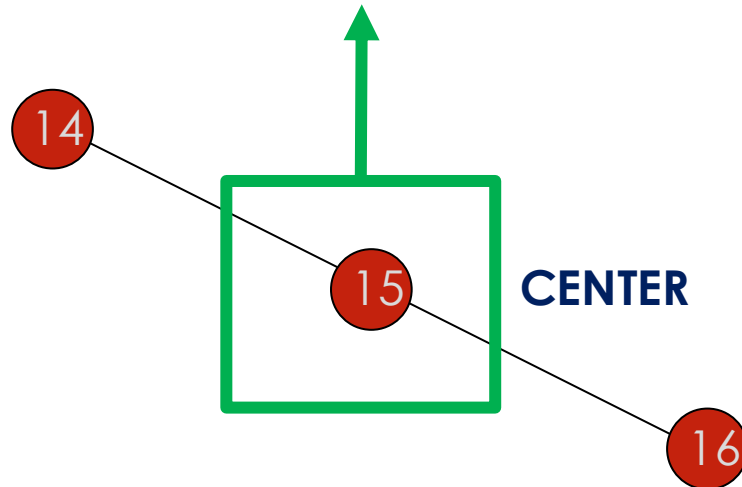


- Need to rotate.

AVL TREE INSERTION –EXAMPLE 1 (CONTD..)

Single rotations: RR Rotation

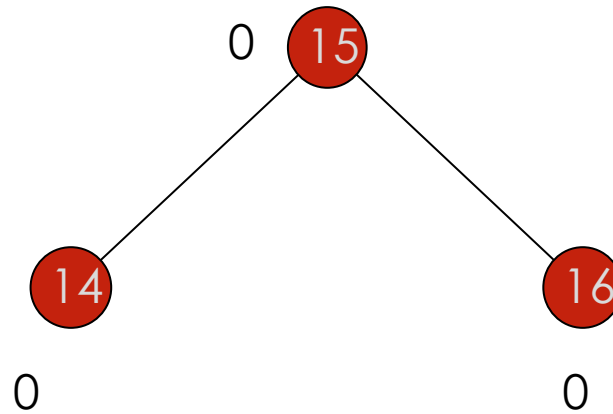
- Rotation type:



AVL TREE INSERTION –EXAMPLE 1 (CONTD..)

Single rotations:

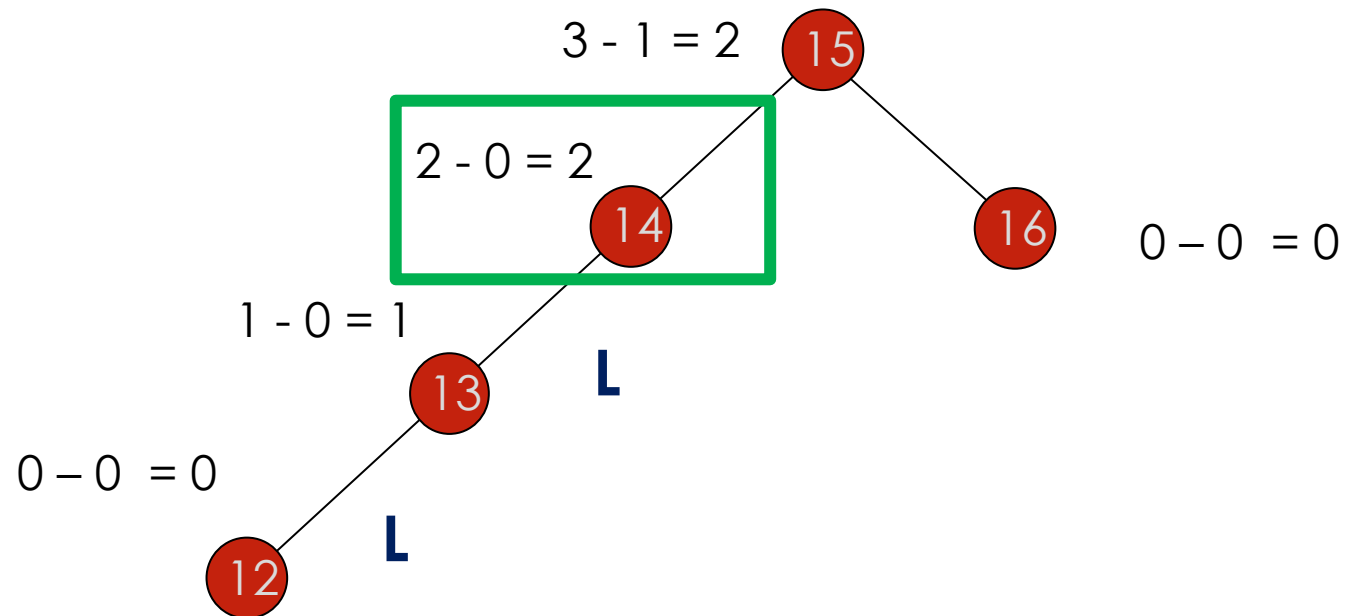
- Rotation restores AVL balance:



AVL TREE INSERTION –EXAMPLE 1 (CONTD..)

Single rotations:

- Now insert 13 and 12:

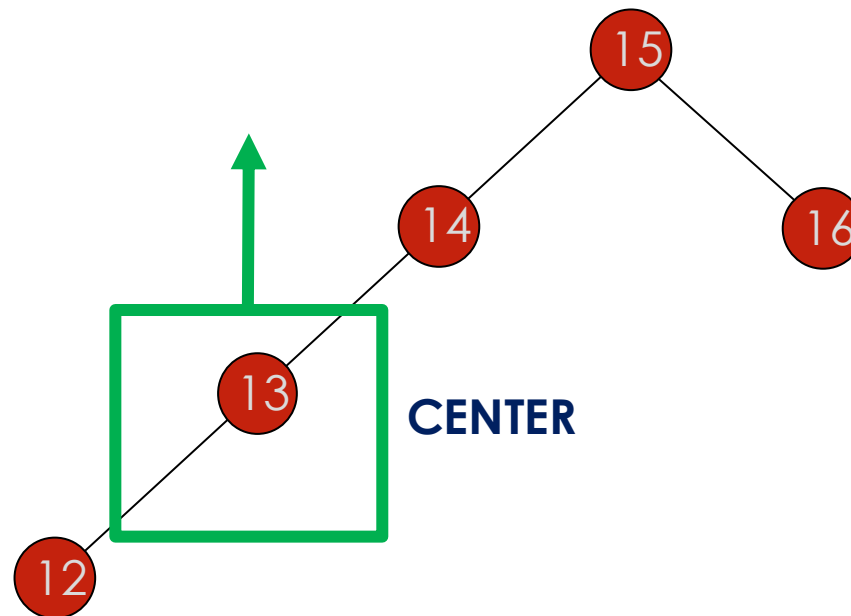


- AVL violation - need to rotate.

AVL TREE INSERTION –EXAMPLE 1 (CONTD..)

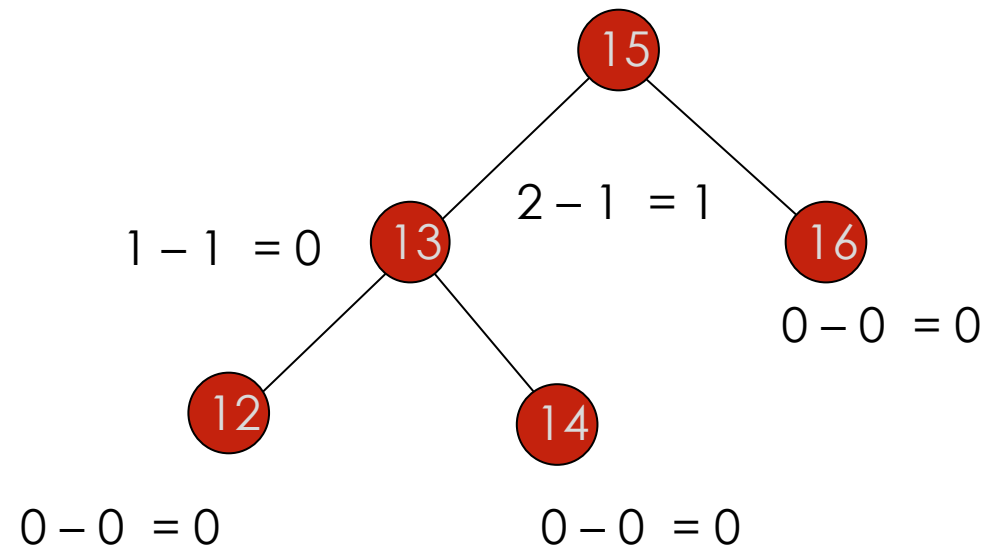
Single rotations: LL

- Rotation type:



AVL TREE INSERTION –EXAMPLE 1 (CONTD..)

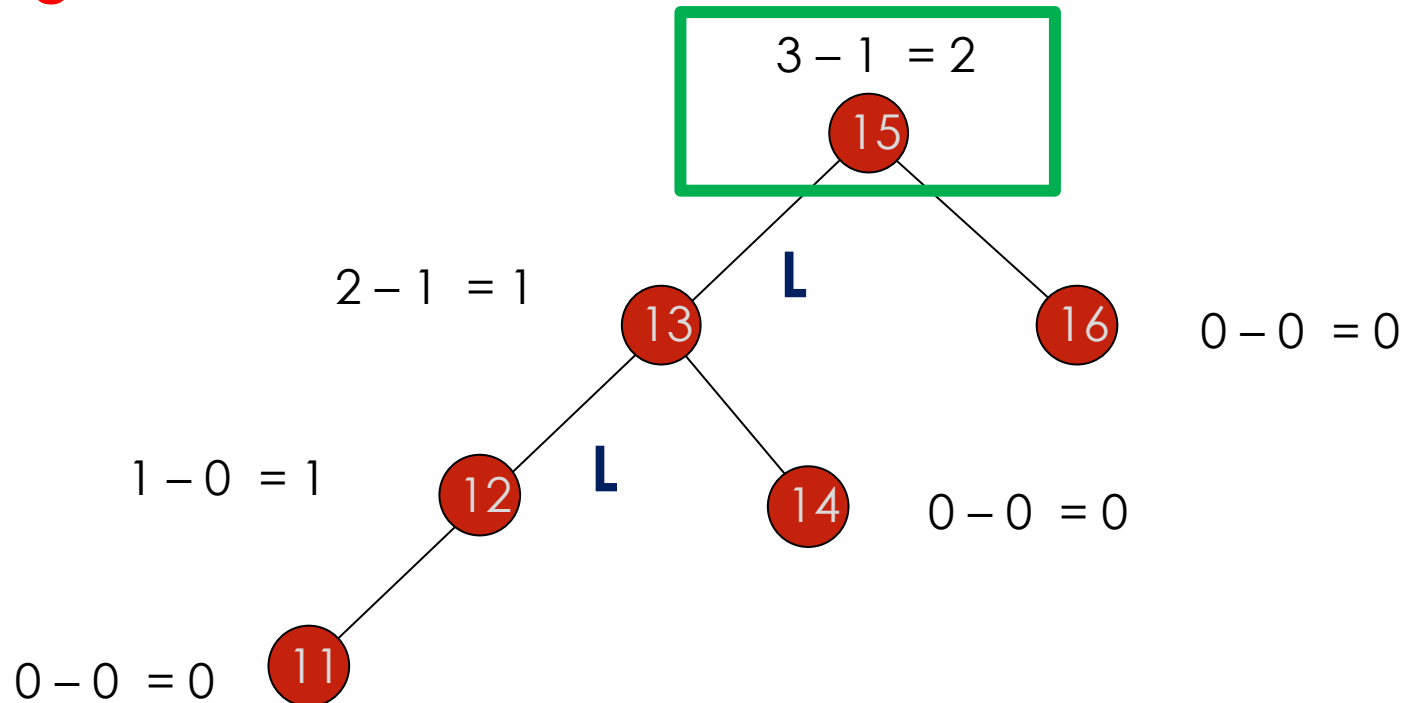
Single rotations:



- Now insert 11.

AVL TREE INSERTION –EXAMPLE 1 (CONTD..)

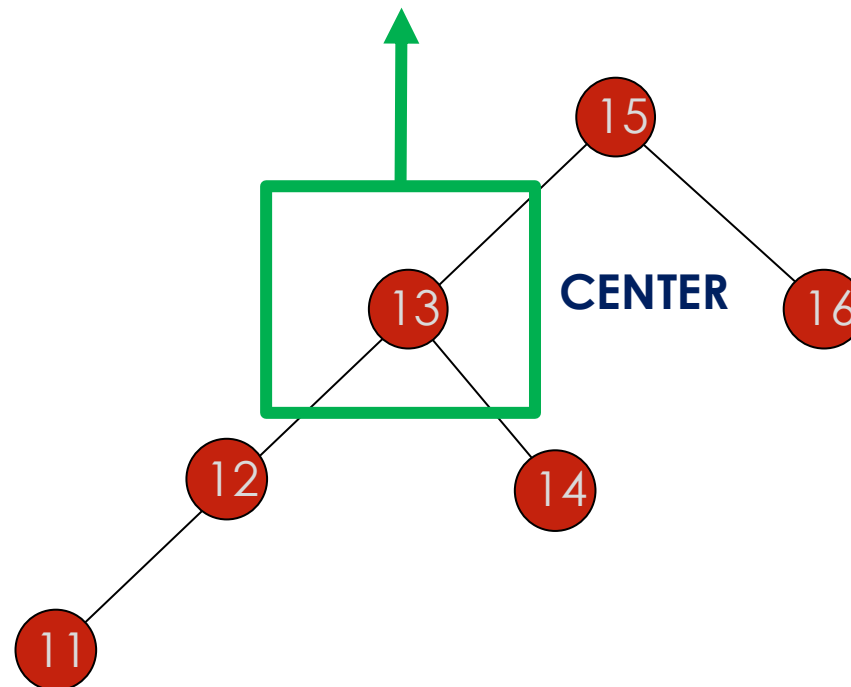
Single rotations:



- AVL violation – need to rotate

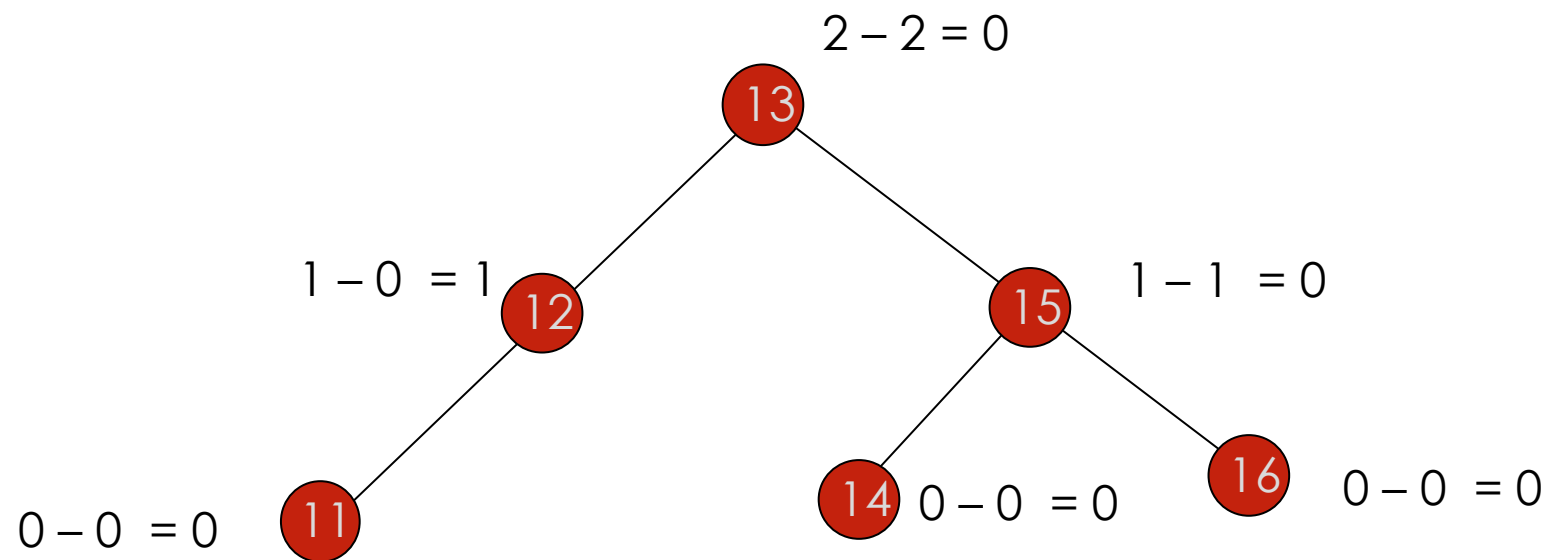
AVL TREE INSERTION –EXAMPLE 1 (CONTD..)

Single rotations: LL



AVL TREE INSERTION –EXAMPLE 1 (CONTD..)

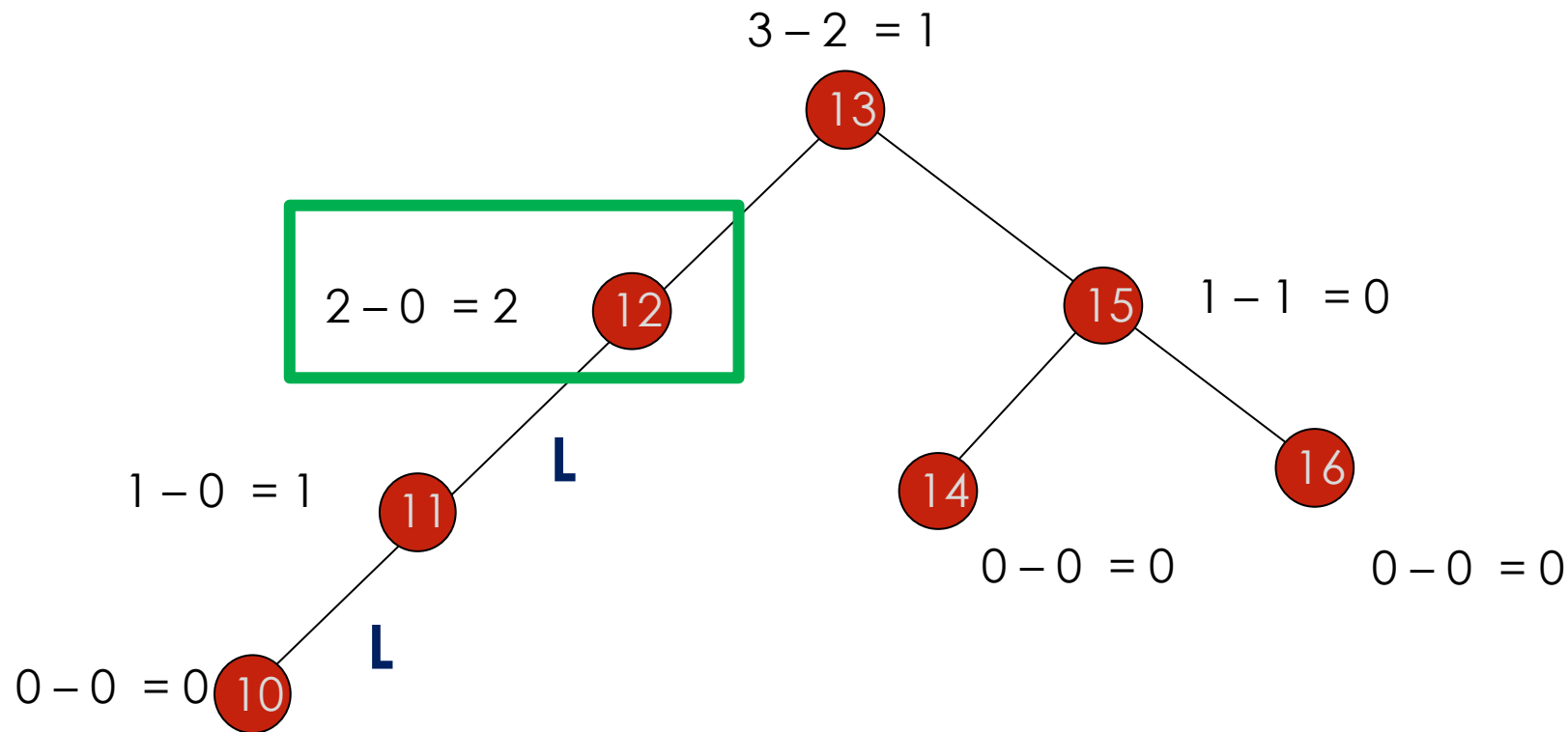
Single rotations:



- Now insert 10.

AVL TREE INSERTION –EXAMPLE 1 (CONTD..)

Single rotations:

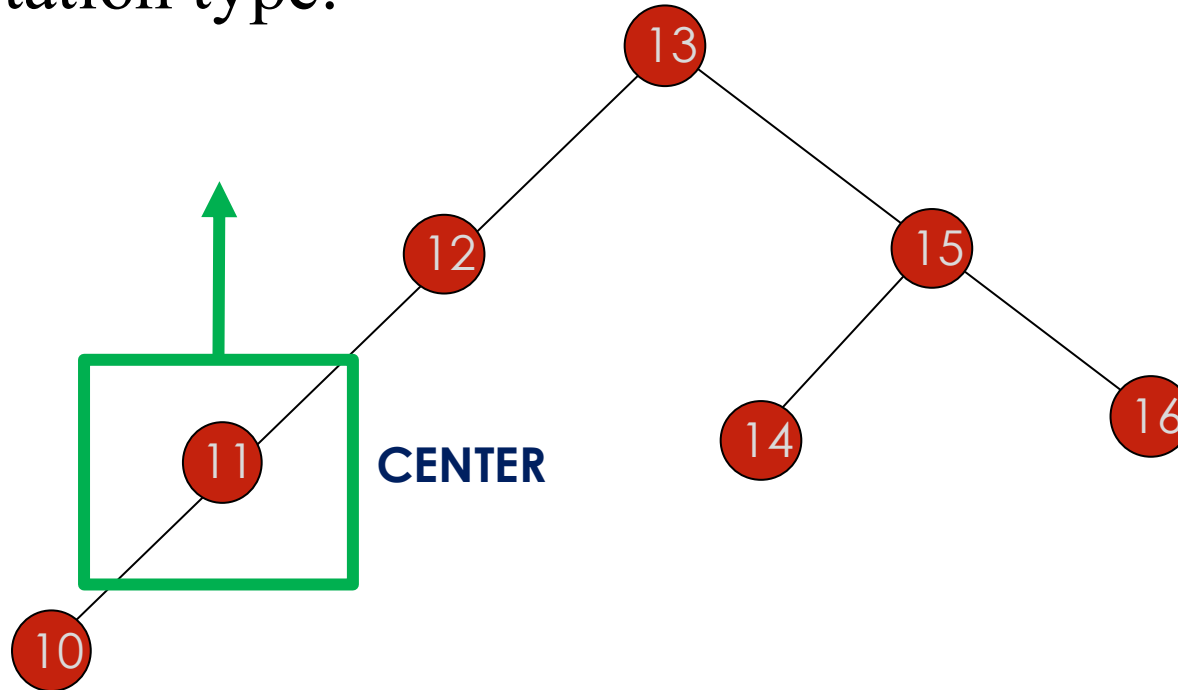


- AVL violation – need to rotate

AVL TREE INSERTION –EXAMPLE 1 (CONTD..)

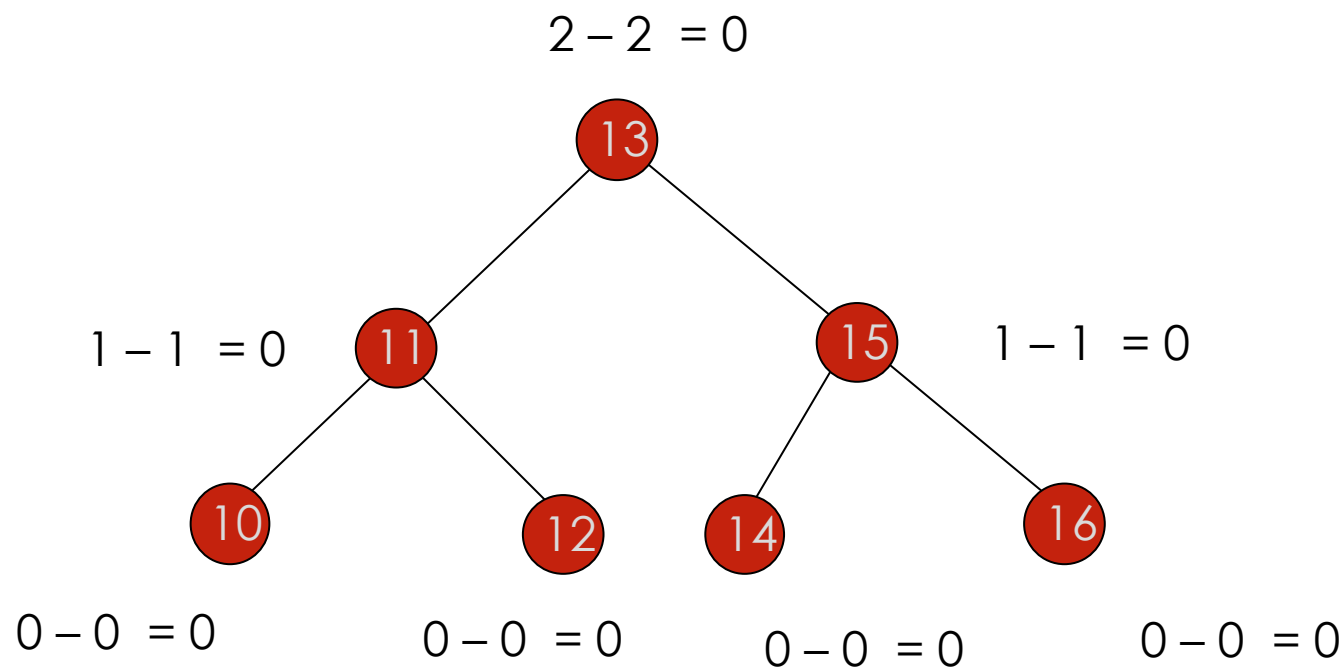
Single rotations: LL

- Rotation type:



AVL TREE INSERTION –EXAMPLE 1 (CONTD..)

Single rotations:



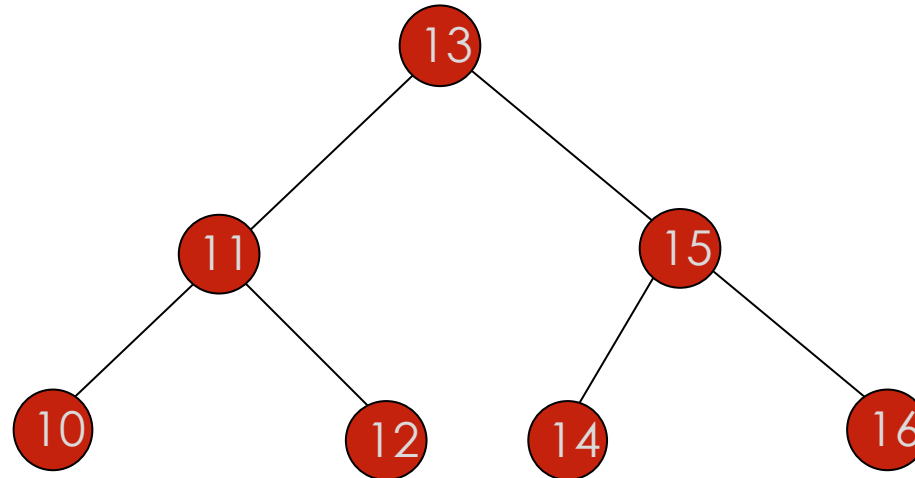
- AVL balance restored.

AVL TREE INSERTION EXAMPLE 2

AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS]

Double rotations: insert 1, 2, 3, 4, 5, 7, 6, 9, 8

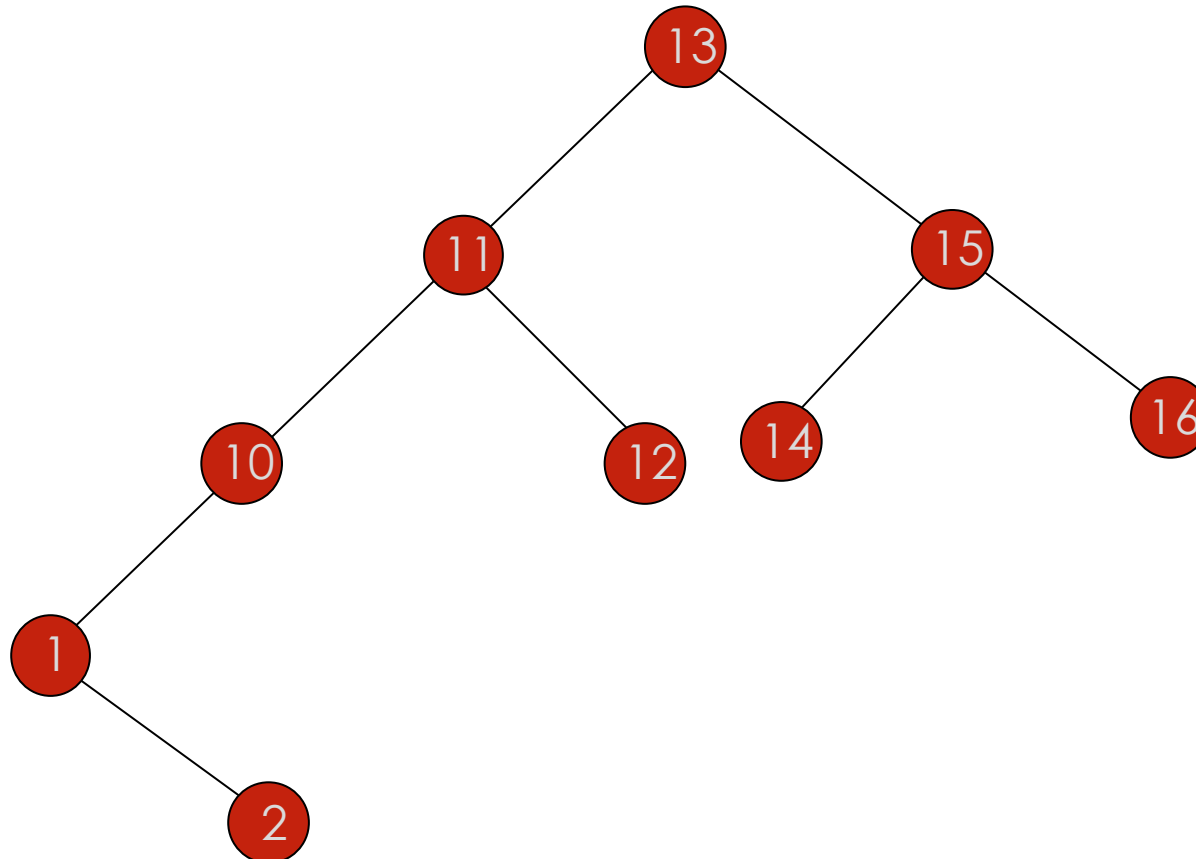
- First insert 1 and 2:



AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

Double rotations:

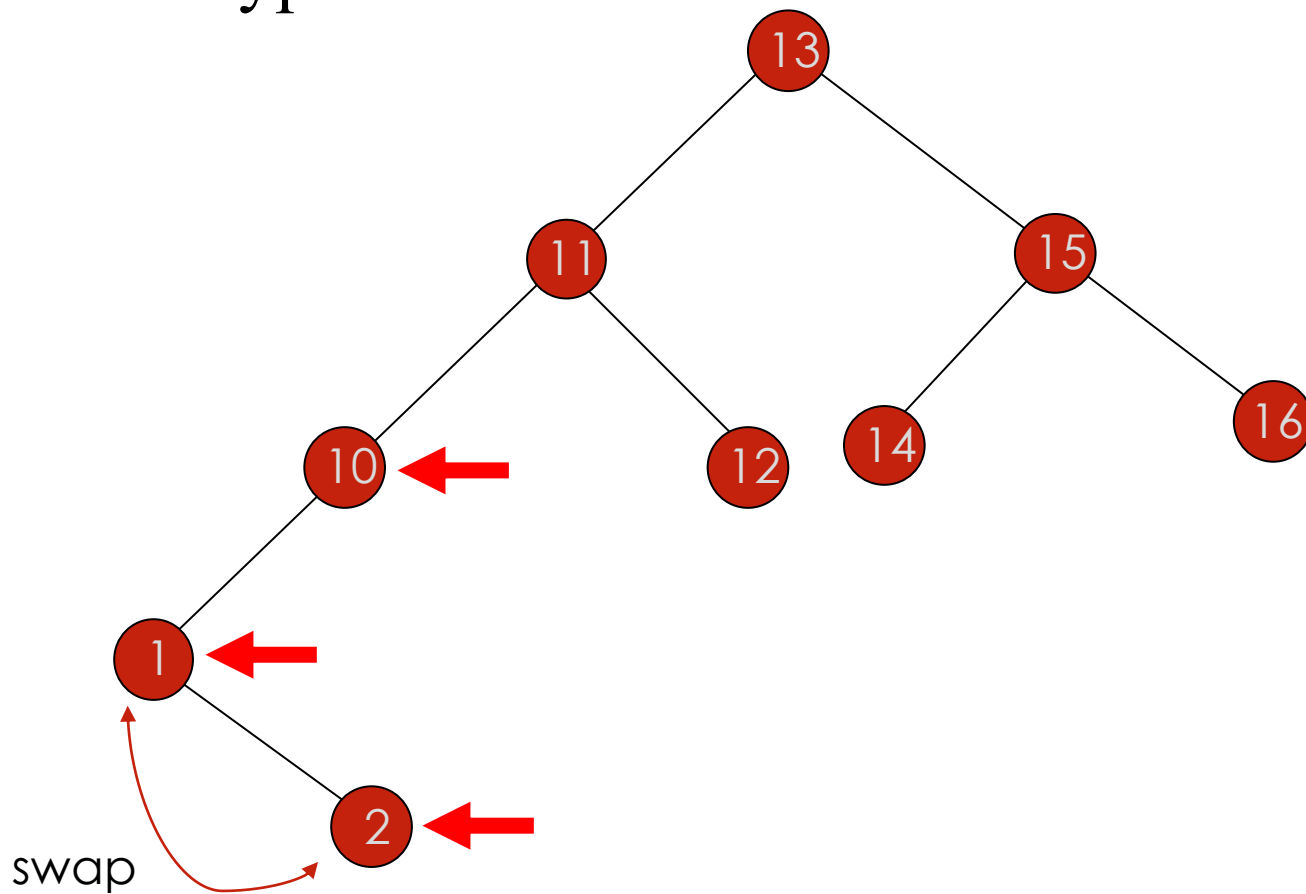
- AVL violation - rotate



AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

Double rotations:

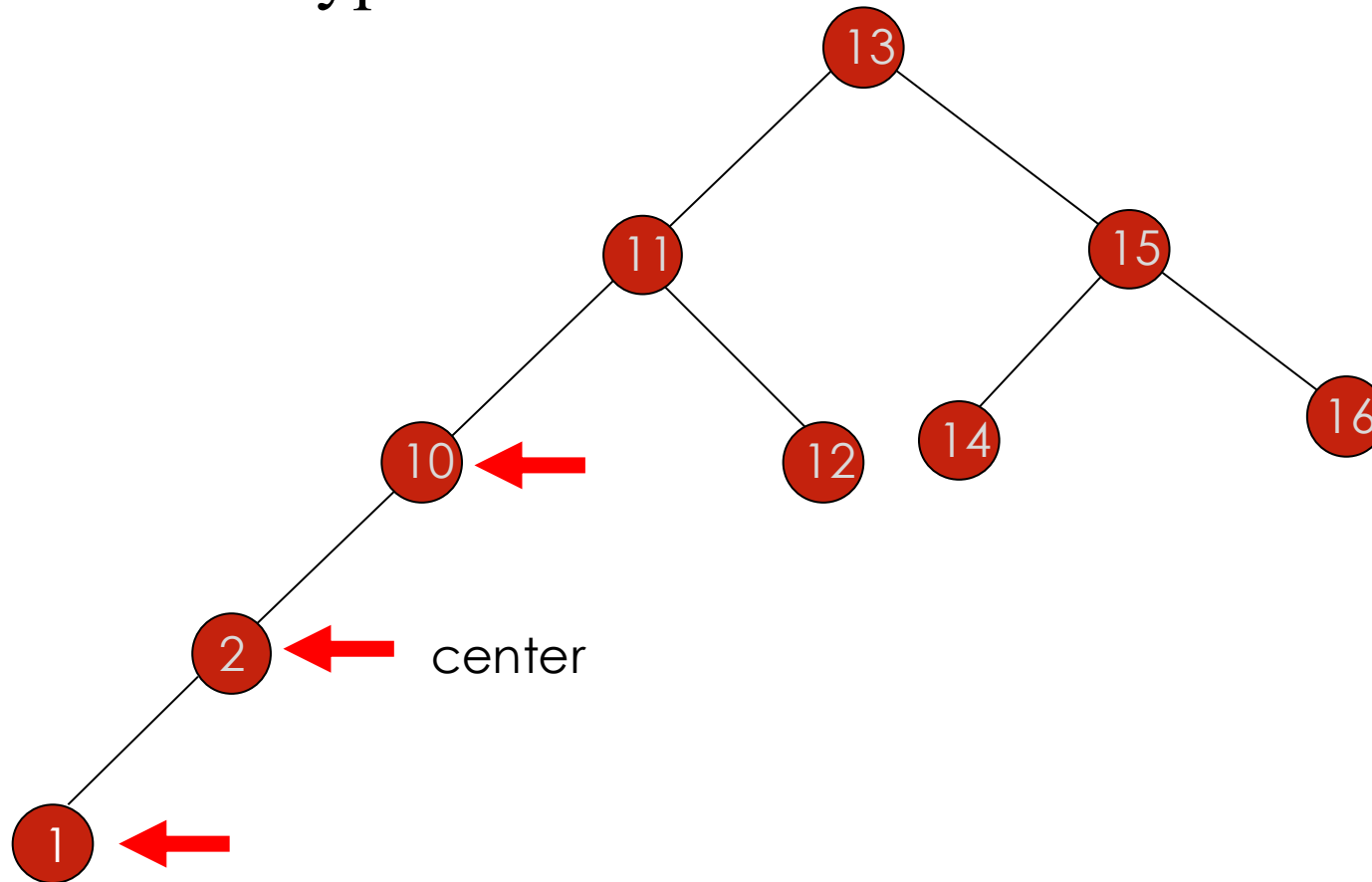
- Rotation type:



AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

Double rotations:

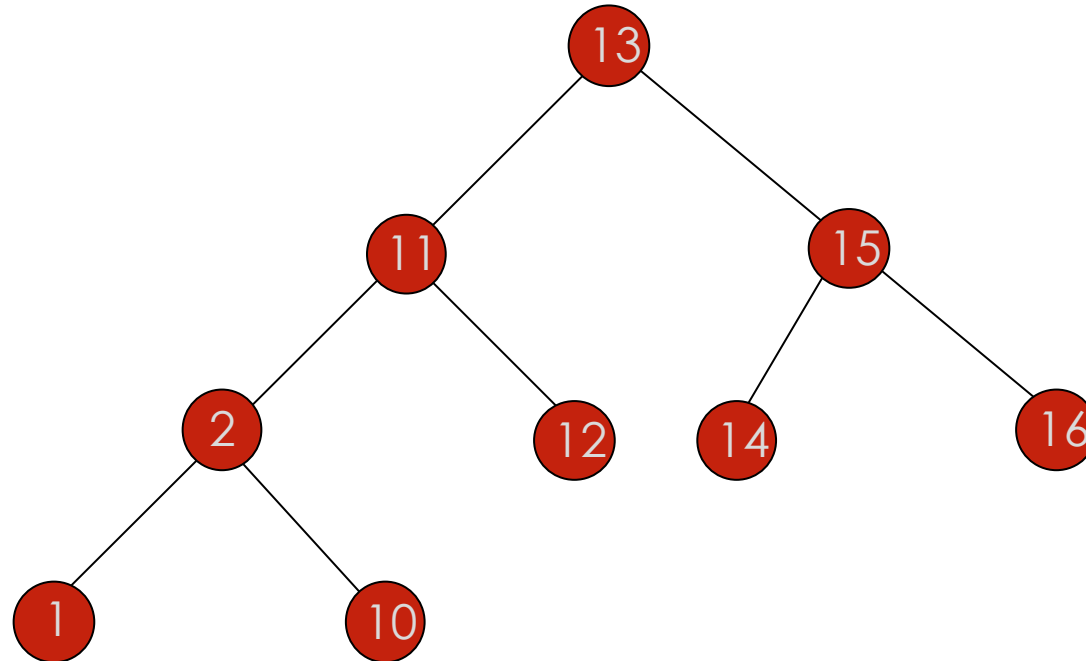
- Rotation type:



AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

Double rotations:

- AVL balance restored:

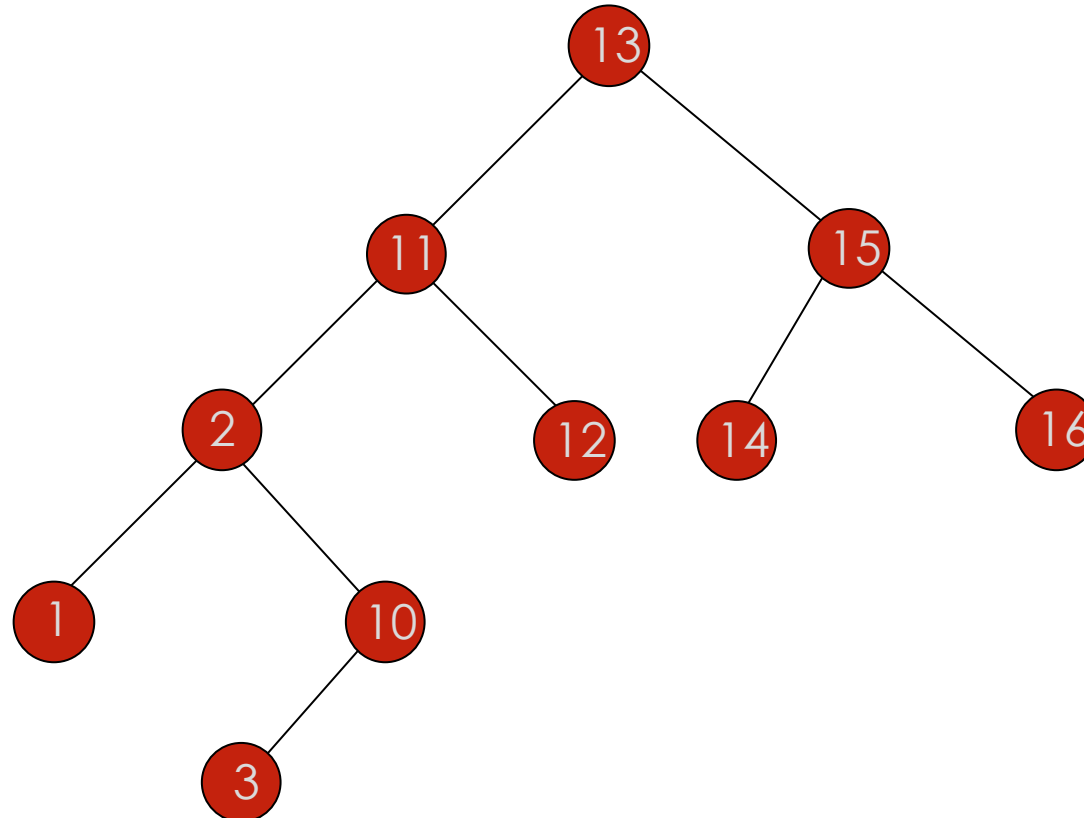


- Now insert 3.

AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

Double rotations:

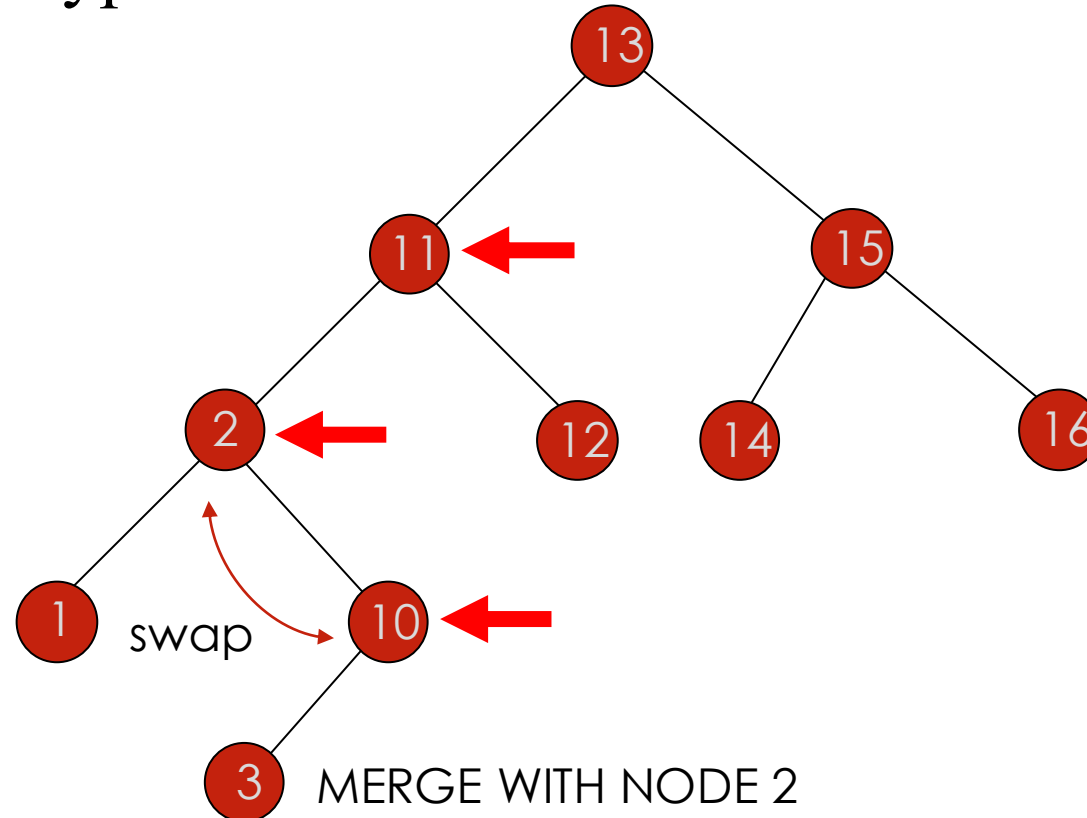
- AVL violation – rotate:



AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

Double rotations:

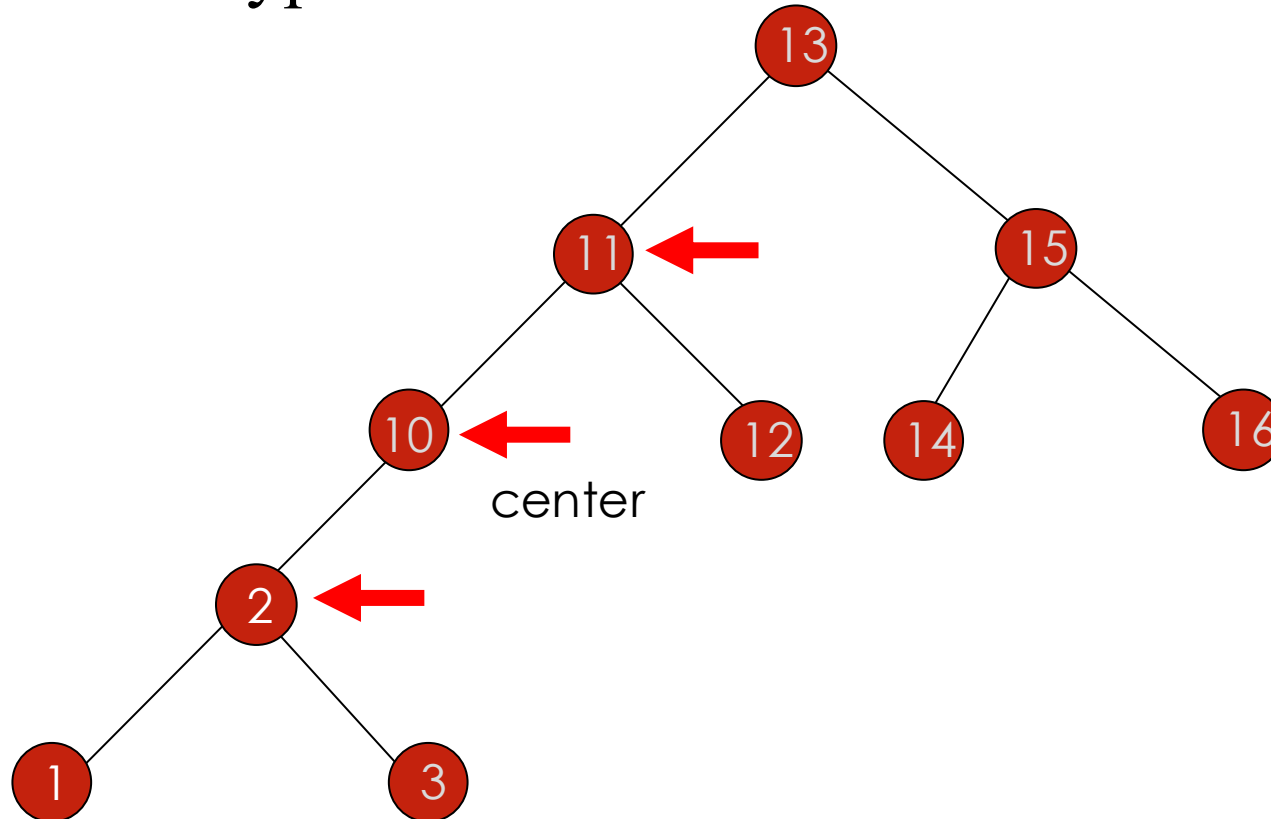
- Rotation type:



AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

Double rotations:

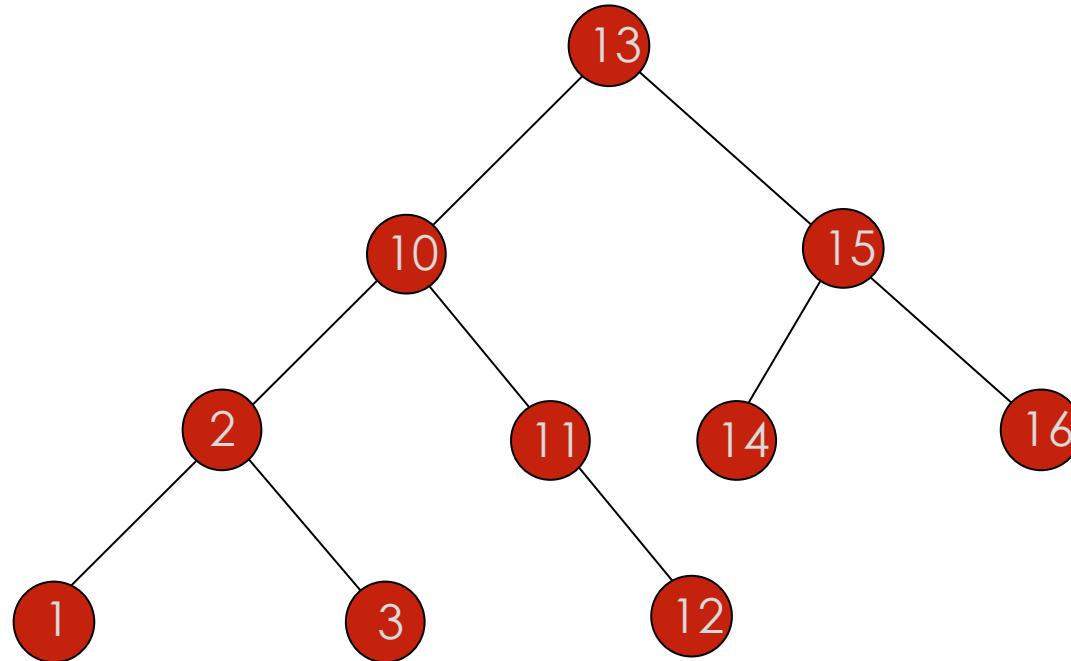
- Rotation type:



AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

Double rotations:

- AVL balance restored:

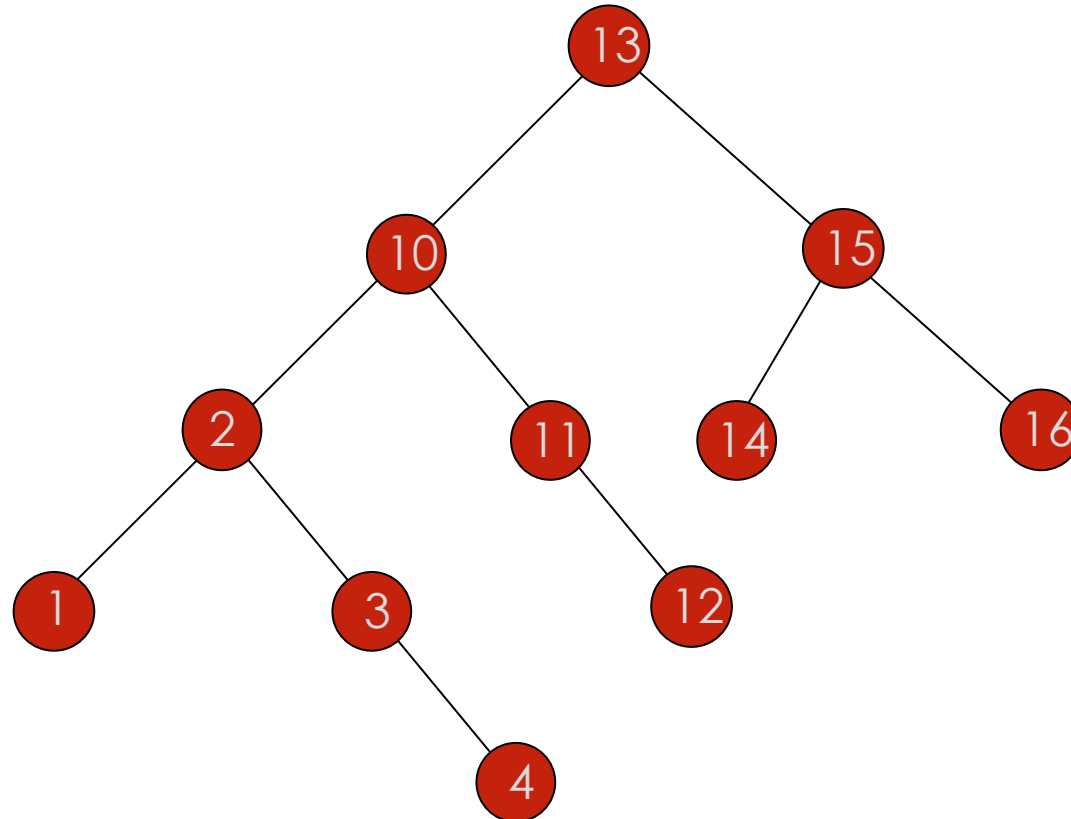


- Now insert 4.

AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

Double rotations:

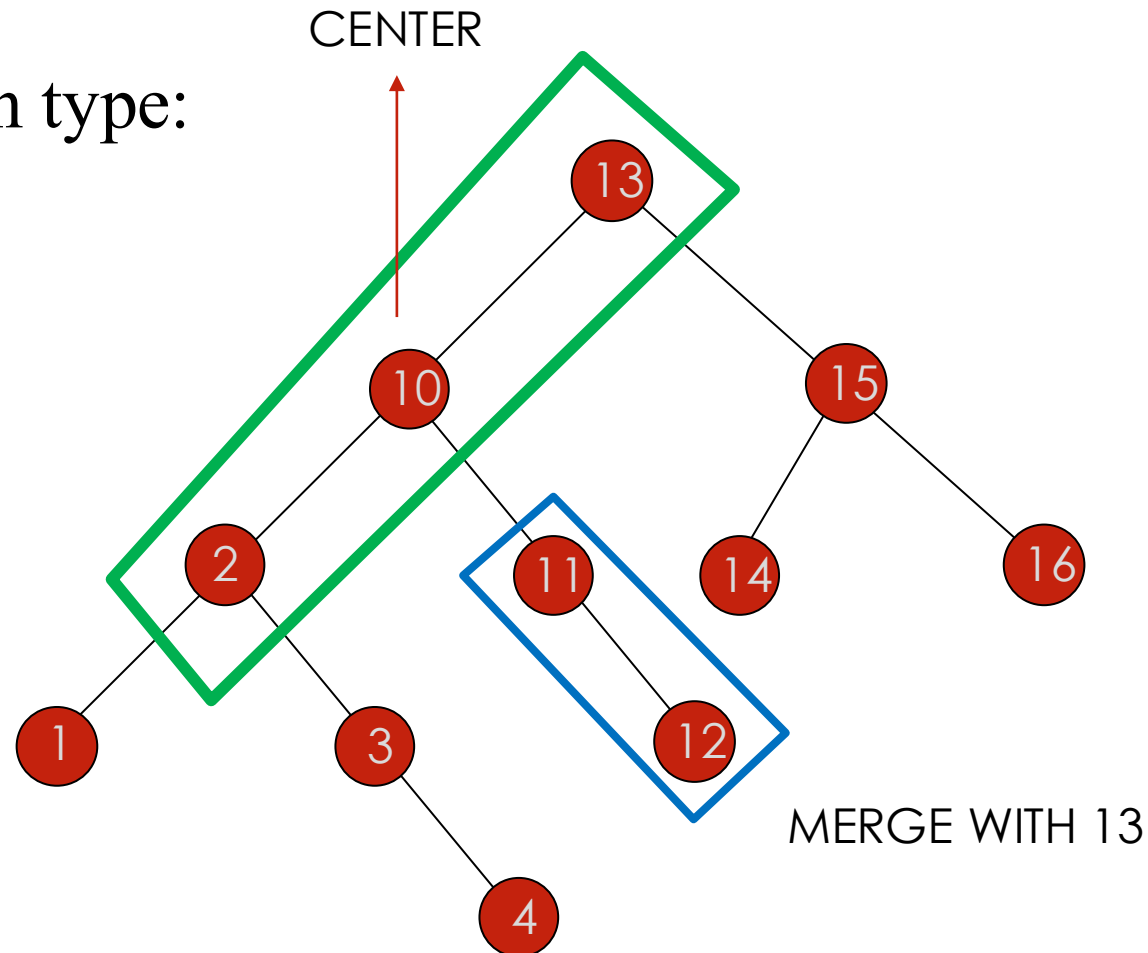
- AVL violation - rotate



AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

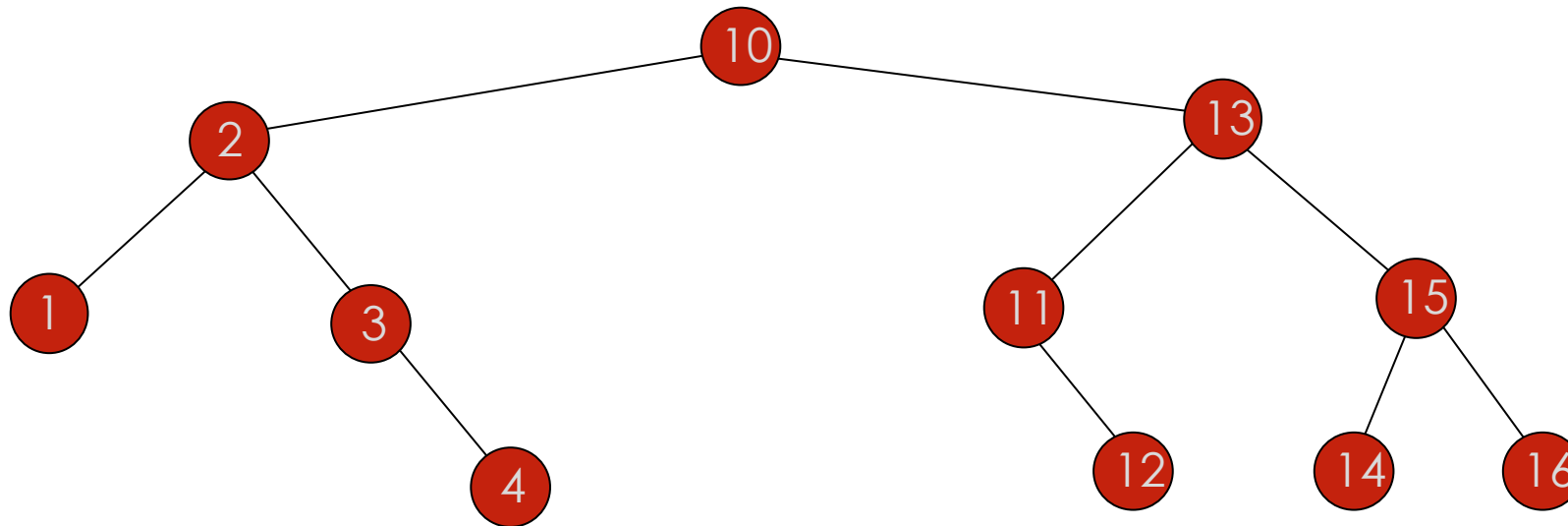
Double rotations:

- Rotation type:



AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

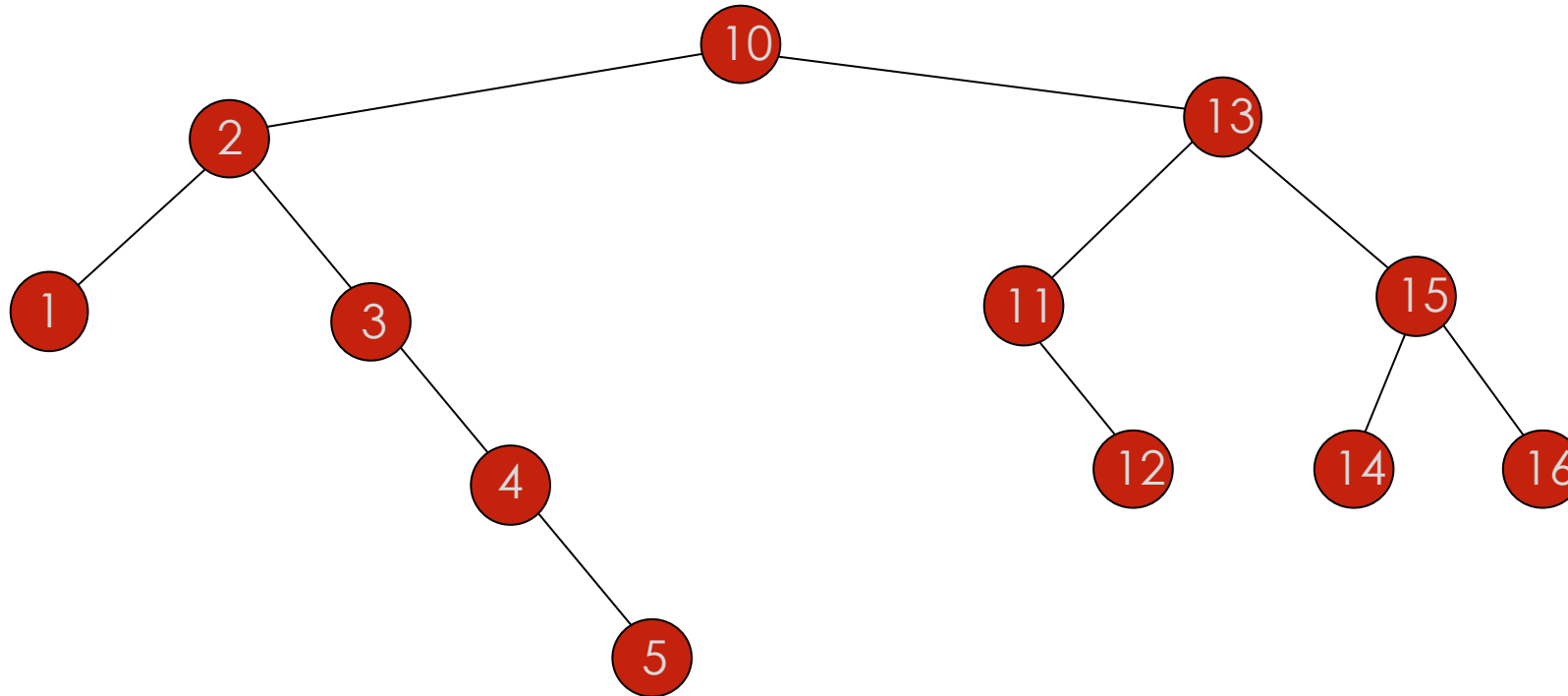
Double rotations:



- Now insert 5.

AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

Double rotations:

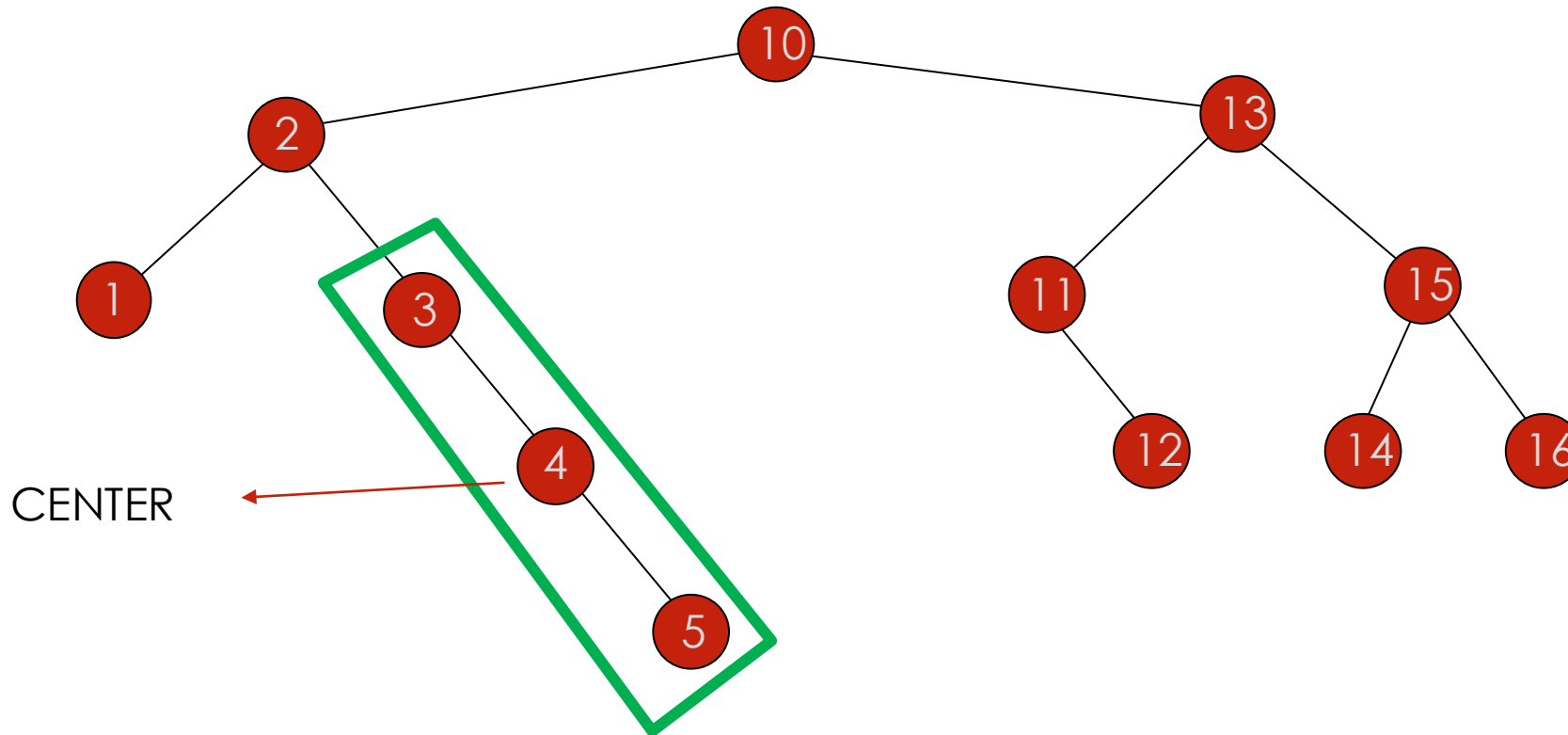


- AVL violation – rotate.

AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

Single rotations:

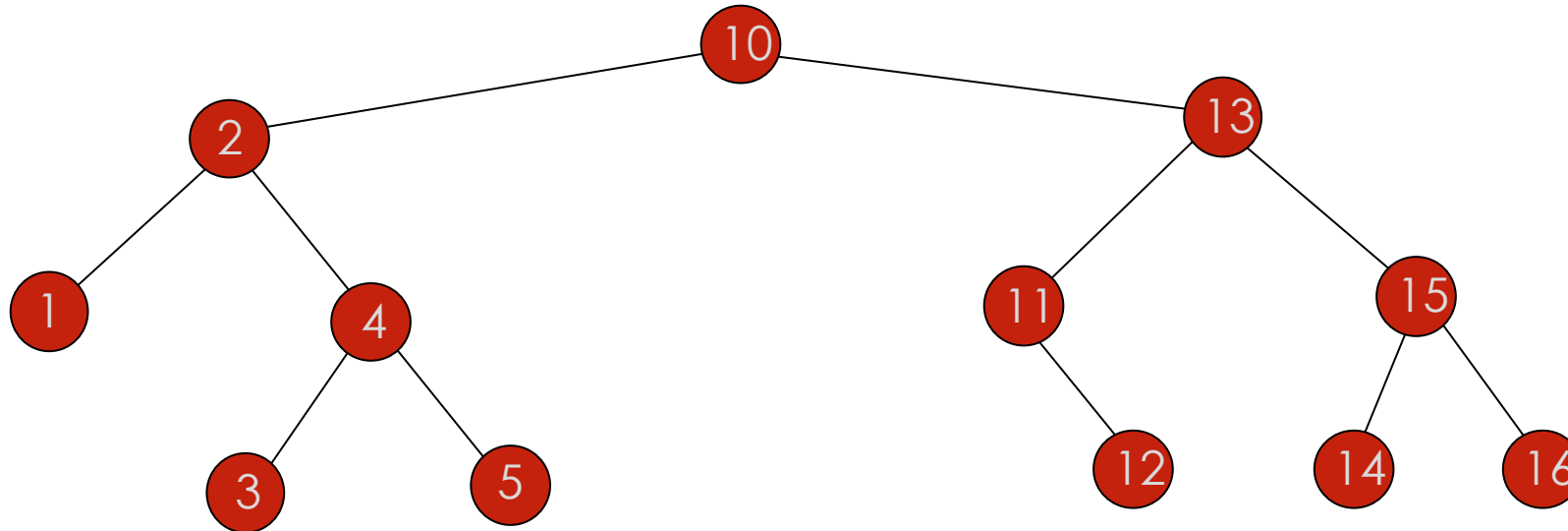
- Rotation type:



AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

Single rotations:

- AVL balance restored:

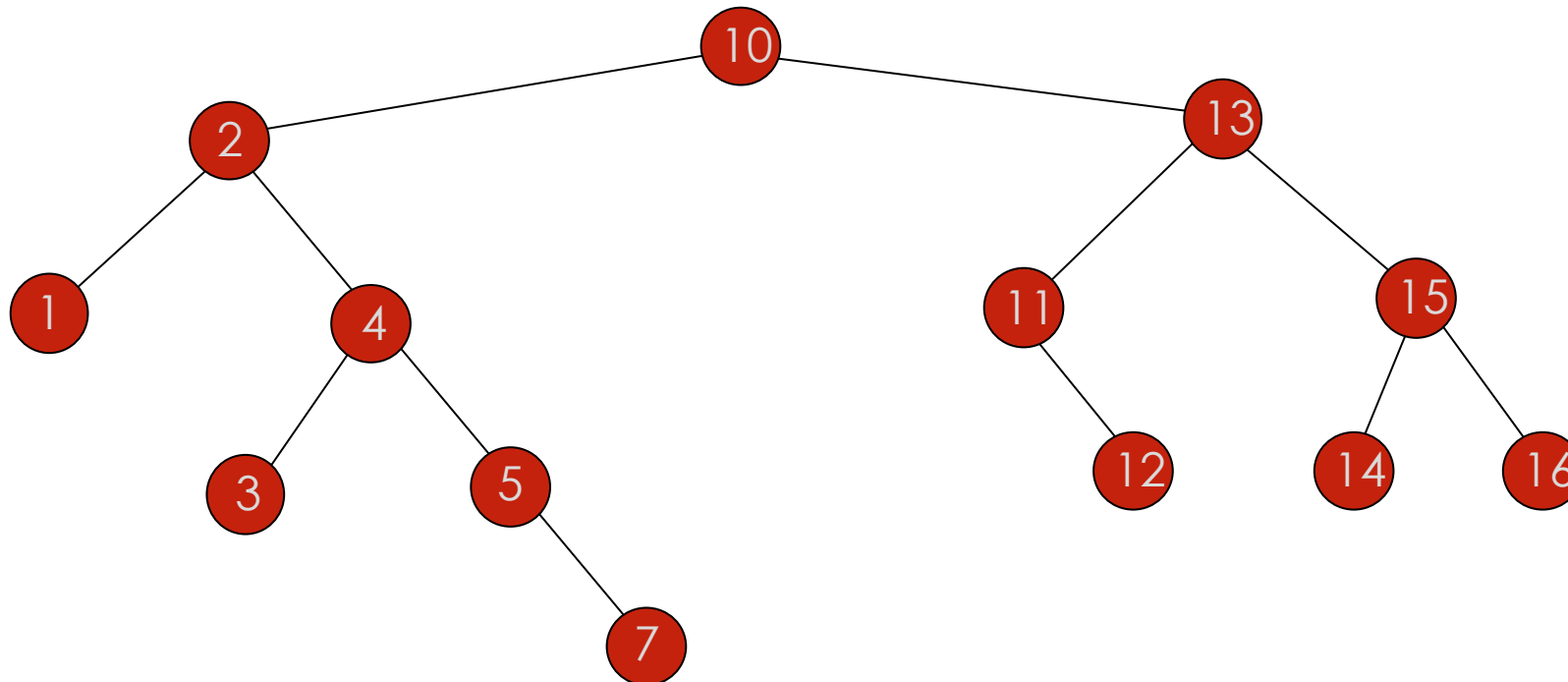


- Now insert 7.

AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

Single rotations:

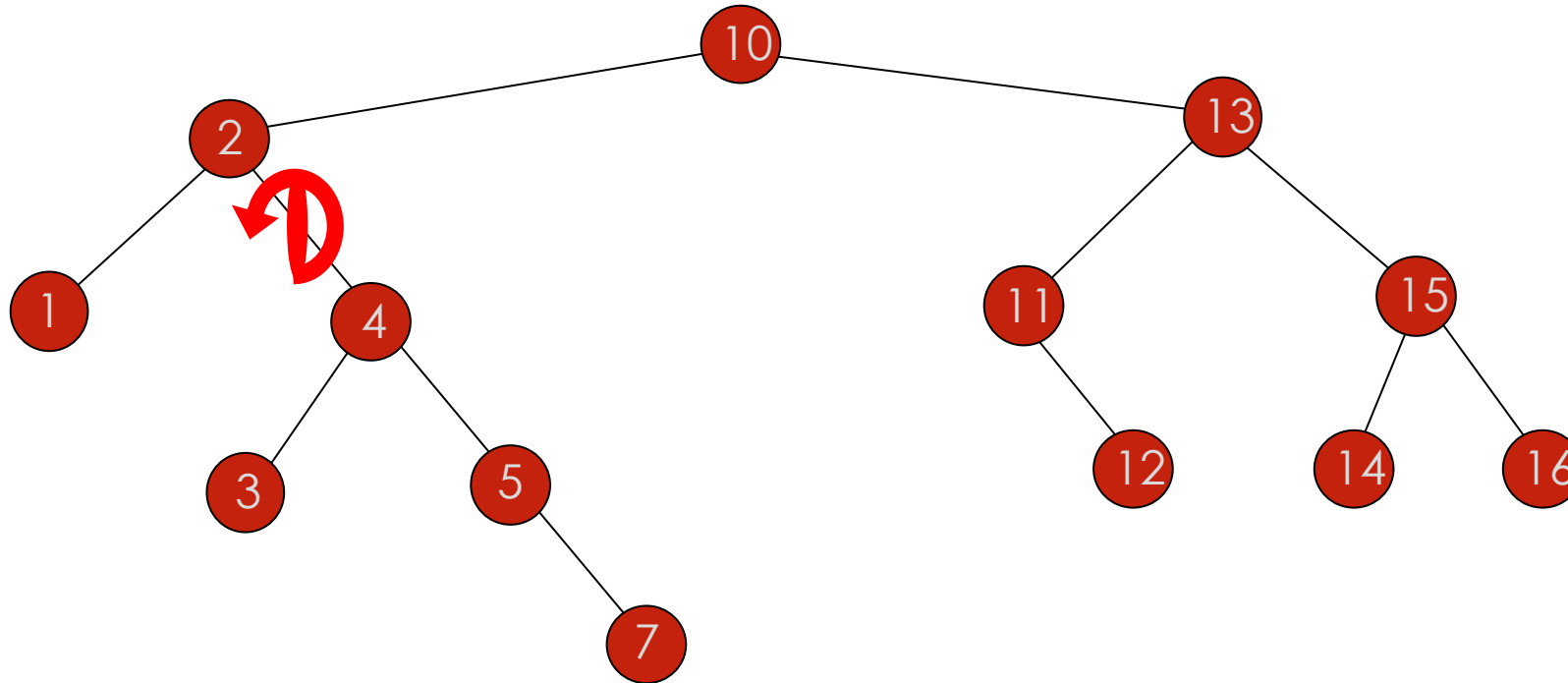
- AVL violation – rotate.



AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

Single rotations:

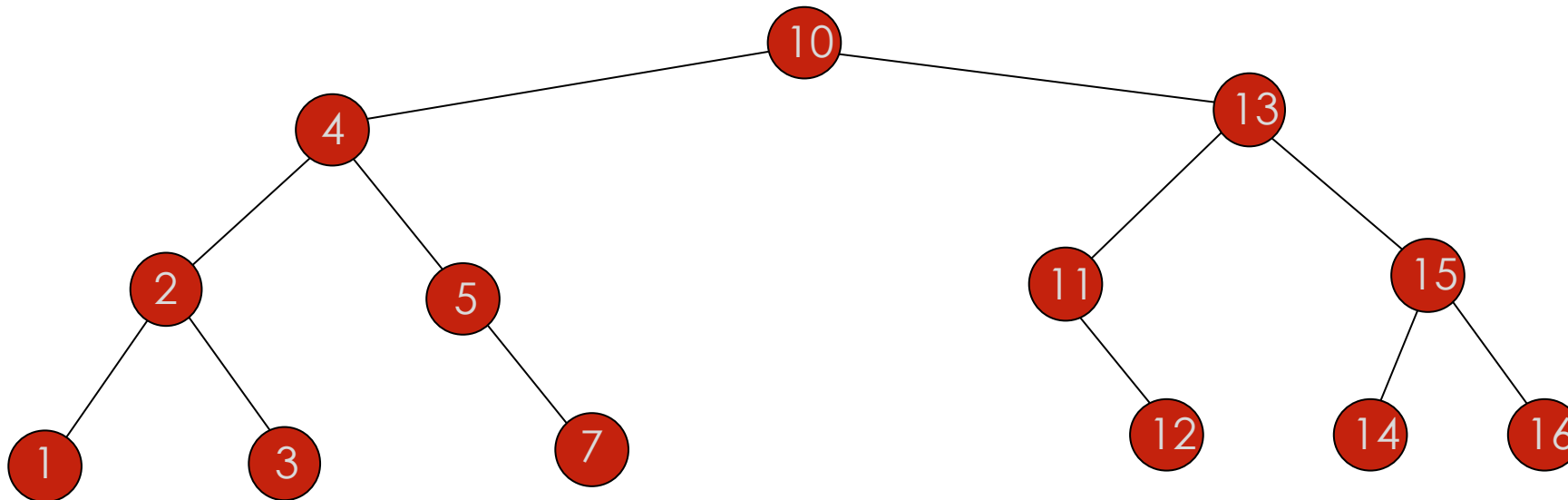
- Rotation type:



AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

Double rotations:

- AVL balance restored.

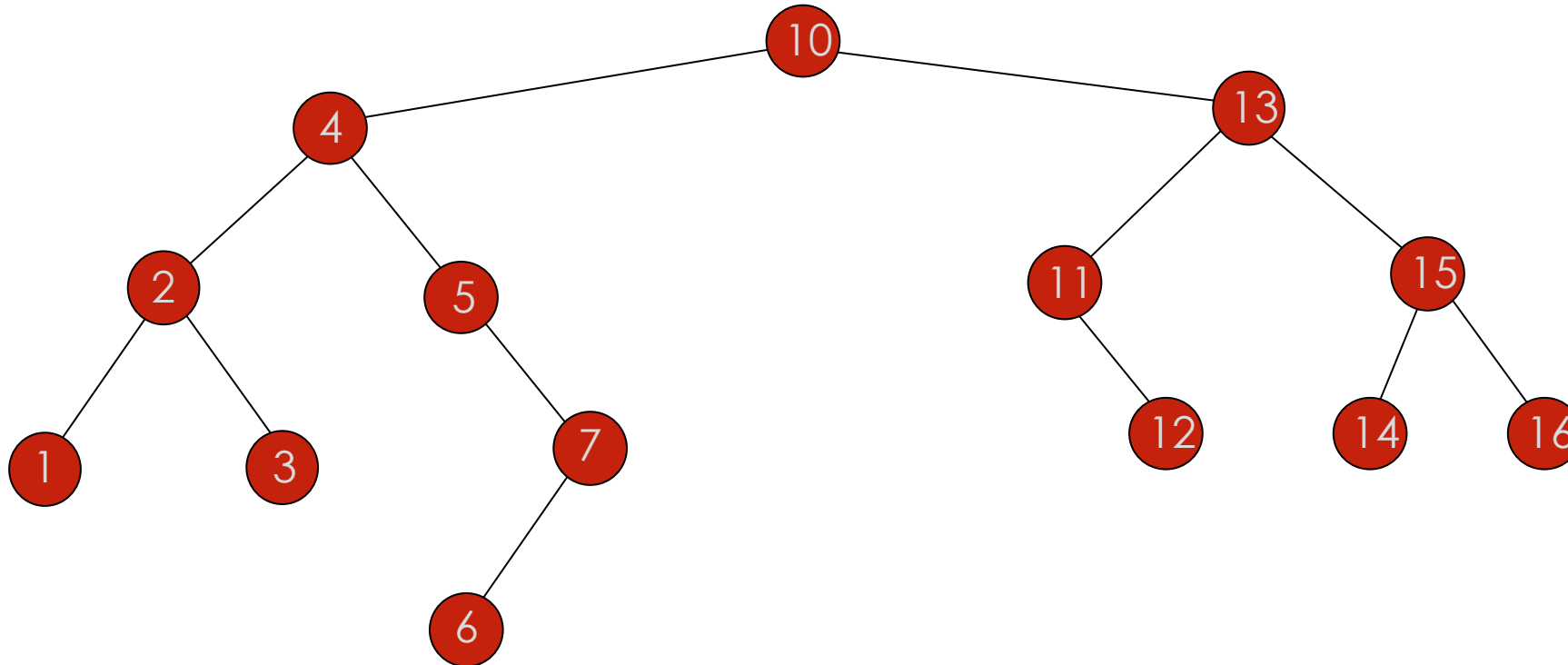


- Now insert 6.

AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

Double rotations:

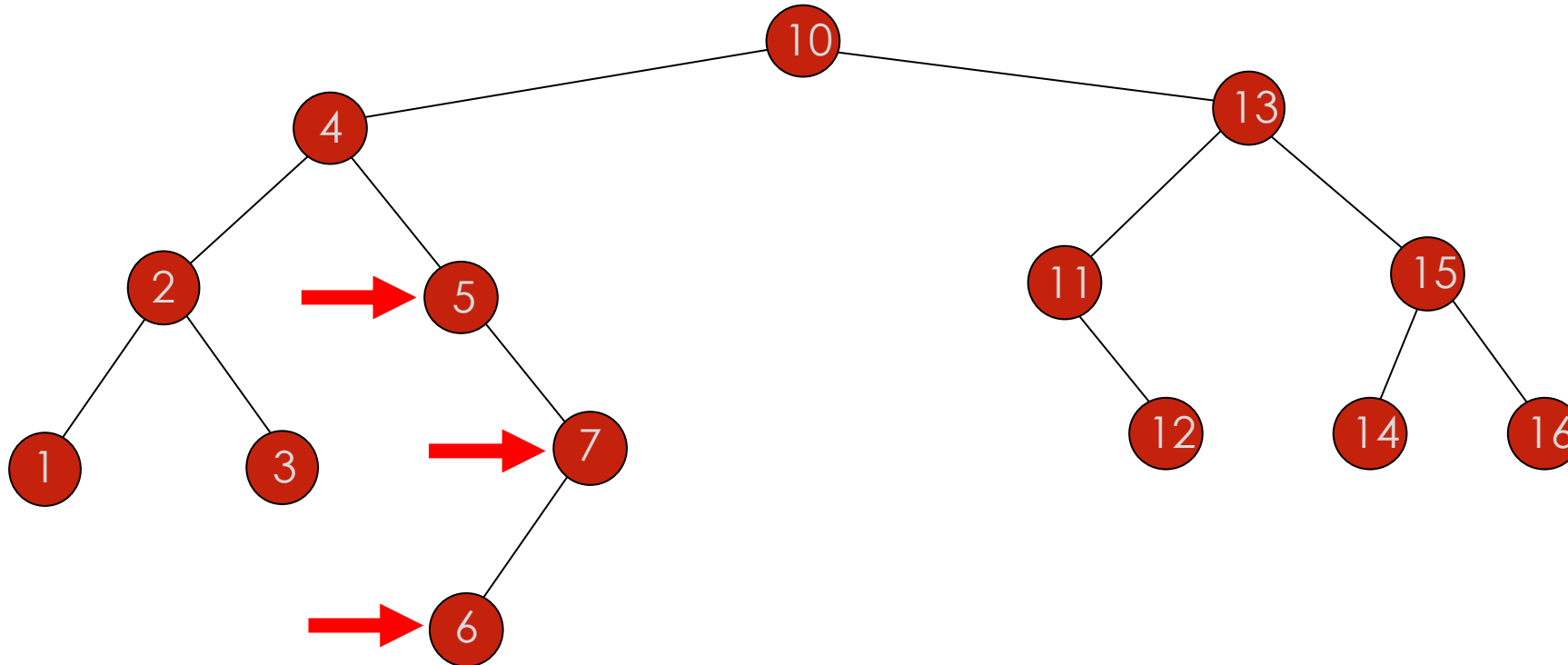
- AVL violation - rotate.



AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

Double rotations:

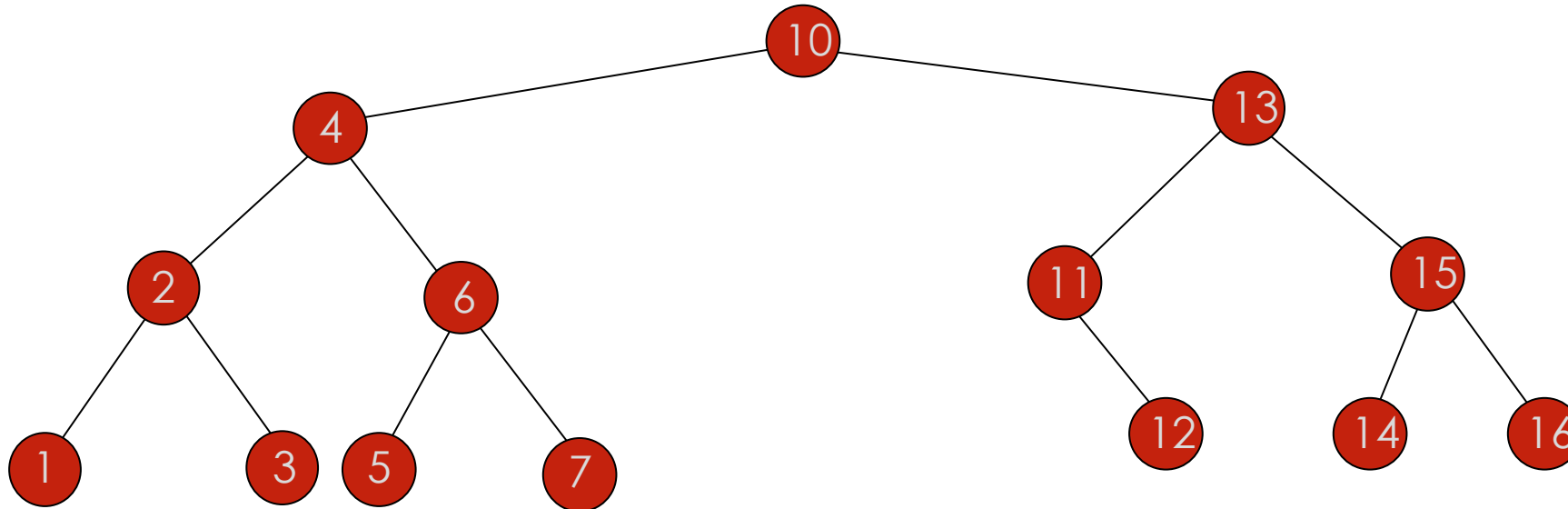
- Rotation type:



AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

Double rotations:

- AVL balance restored.

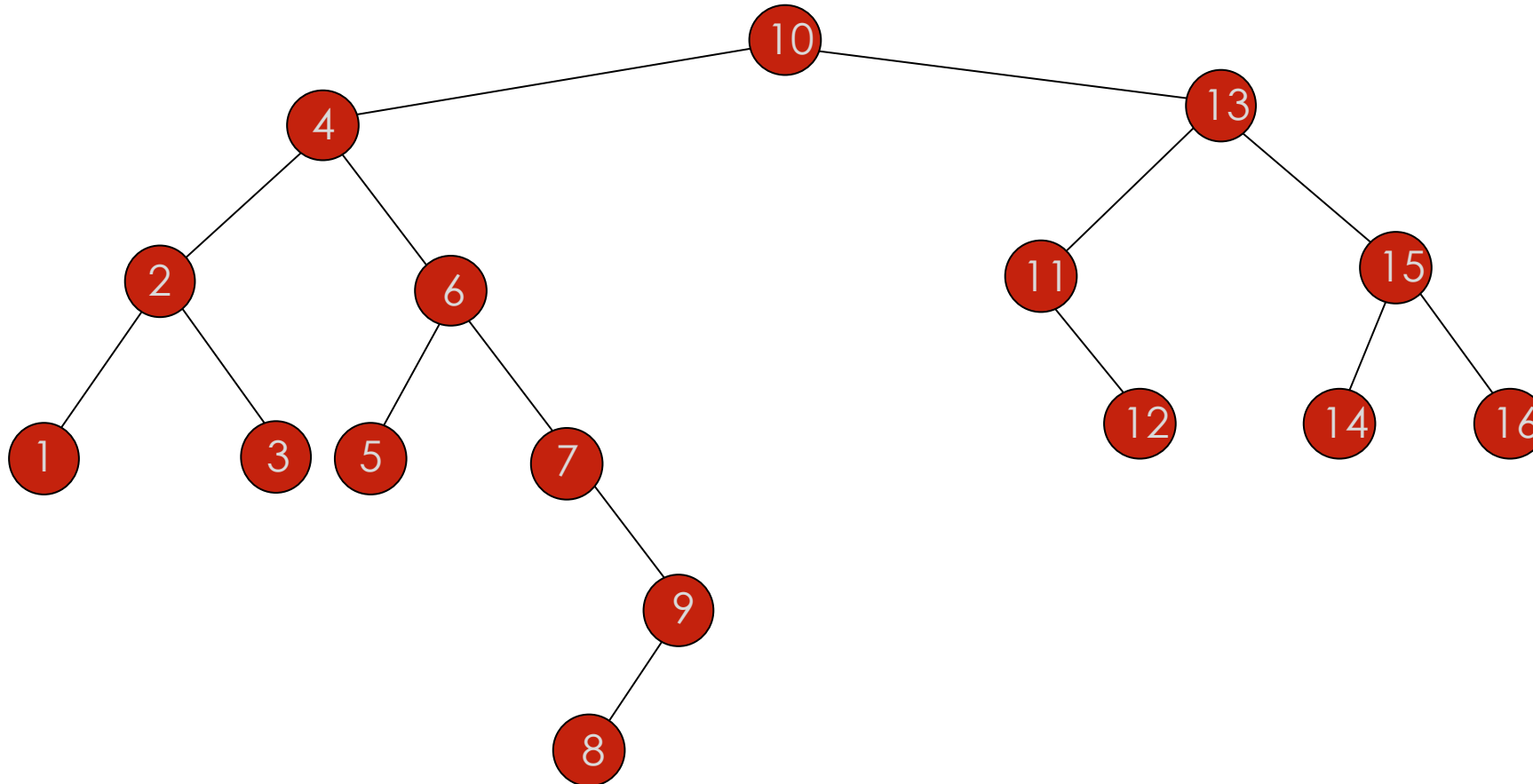


- Now insert 9 and 8.

AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

Double rotations:

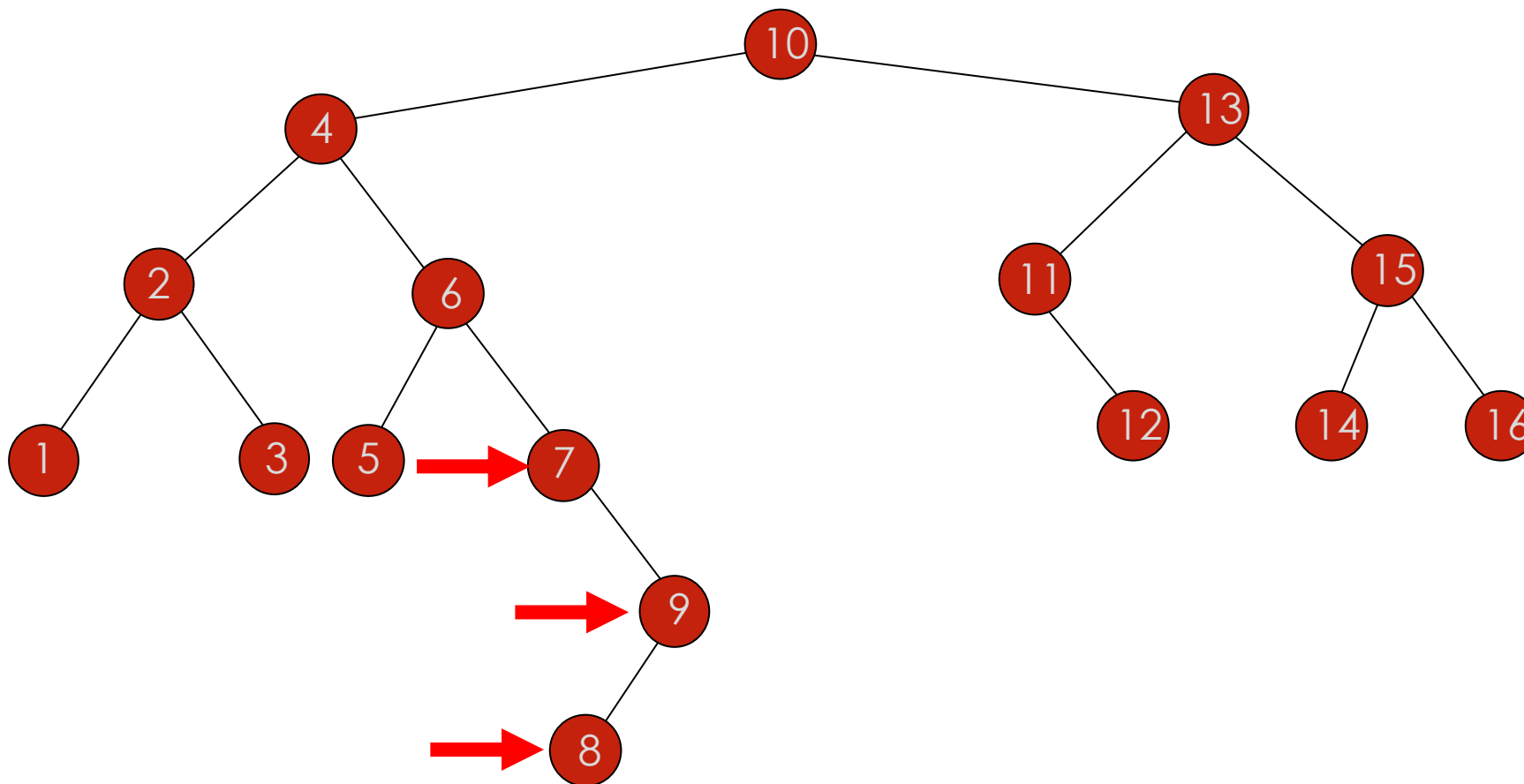
- AVL violation - rotate.



AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

Double rotations:

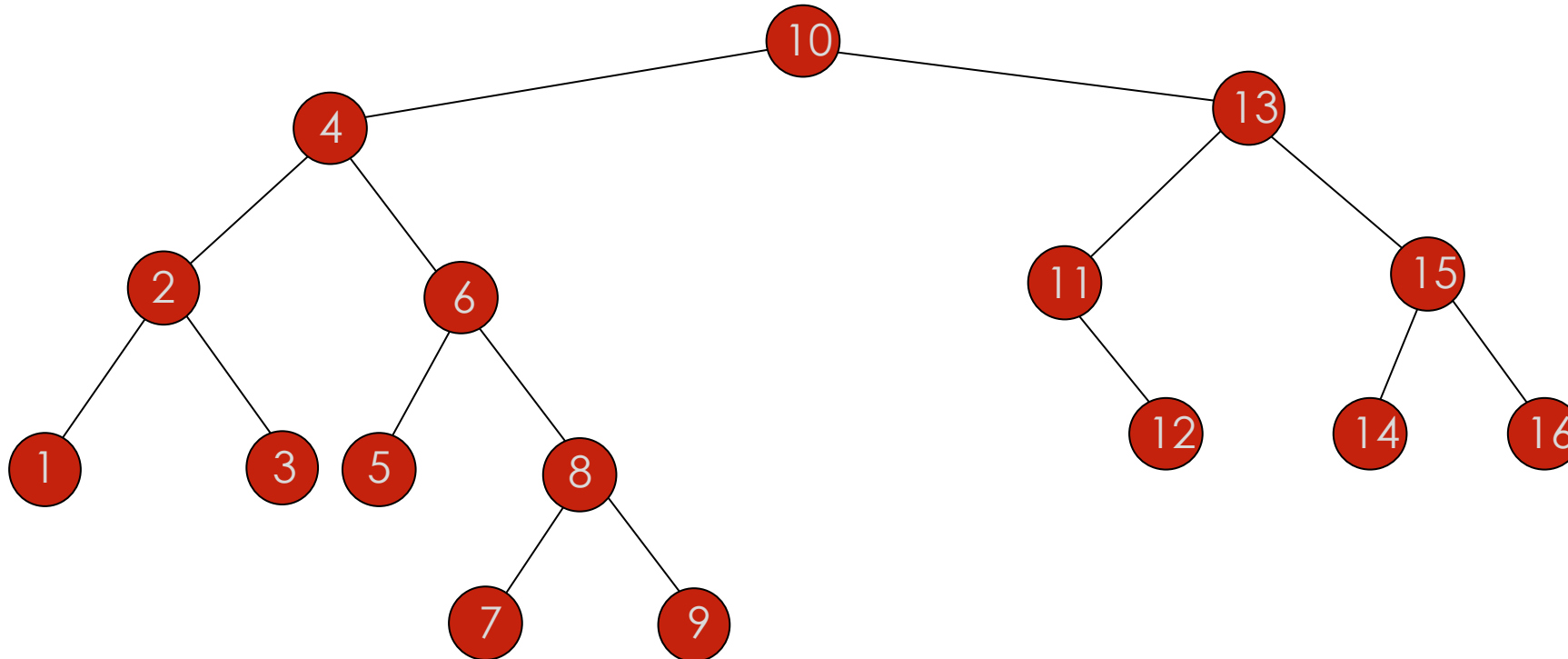
- Rotation type:



AVL TREE INSERTION [EXAMPLE 2: CONSISTS OF SINGLE AND DOUBLE ROTATIONS] (CONTD..)

Final tree:

- Tree is almost perfectly balanced



AVL TREE – DELETION OF NODE

- 1) Perform the normal BST[Binary Search Tree] deletion.
- 2) The current node must be one of the ancestors of the deleted node. Update the height of the current node.
- 3) Get the balance factor (left subtree height – right subtree height) of the current node.

AVL TREE –DELETION OF NODE (CONTD..)

- 4) If balance factor is greater than 1, then the current node is unbalanced and we are either in Left Left case or Left Right case. To check whether it is Left Left case or Left Right case, get the balance factor of left subtree. If balance factor of the left subtree is greater than or equal to 0, then it is Left Left case, else Left Right case.



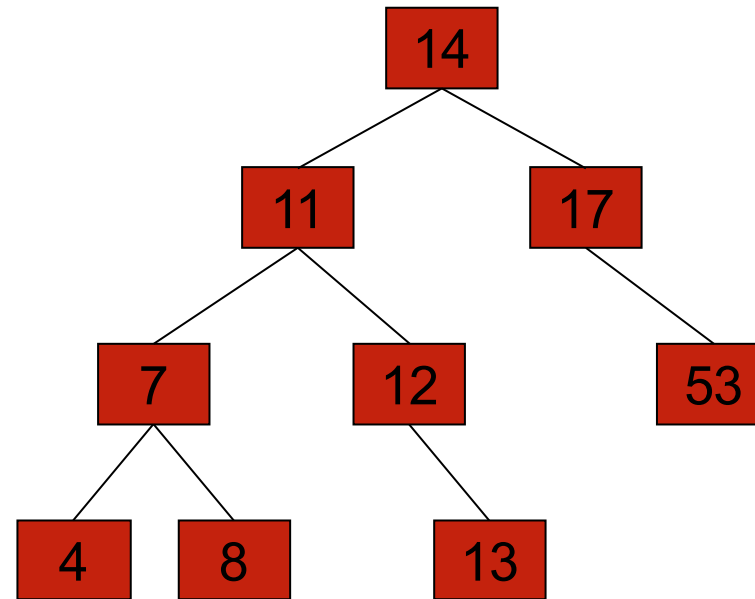
AVL TREE –DELETION OF NODE (CONTD..)

- **5) If balance factor is less than -1, then the current node is unbalanced and we are either in Right Right case or Right Left case. To check whether it is Right Right case or Right Left case, get the balance factor of right subtree. If the balance factor of the right subtree is smaller than or equal to 0, then it is Right Right case, else Right Left case.**



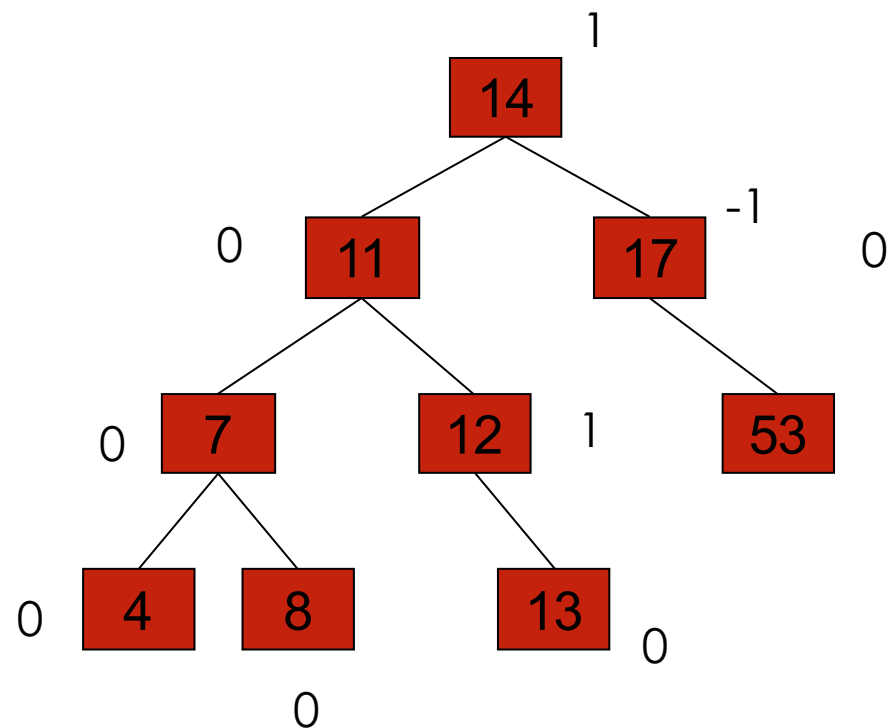
AVL Tree Deletion Example 1:

Remove the nodes 53, 11 and 8



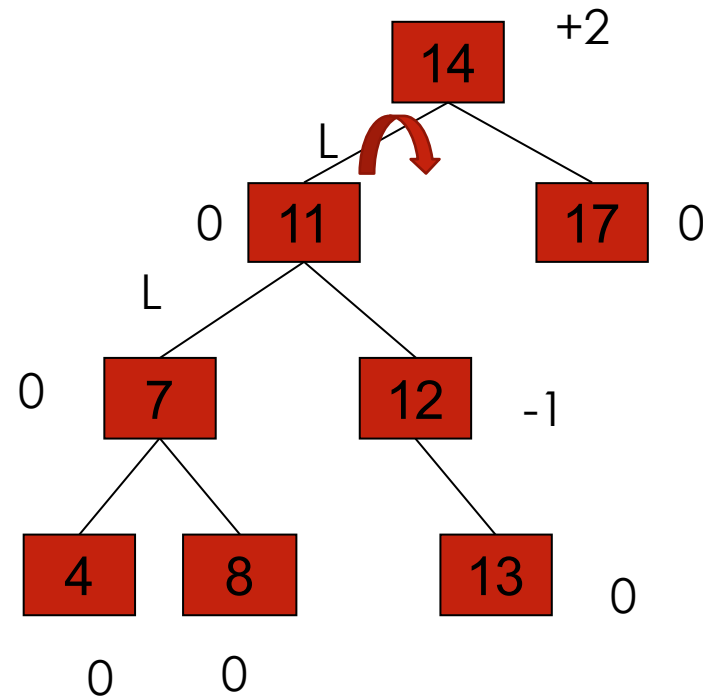
AVL Tree deletion: Example 1 (contd..)

- Now remove 53



AVL Tree Deletion Example 1:

- Now remove 53, unbalanced



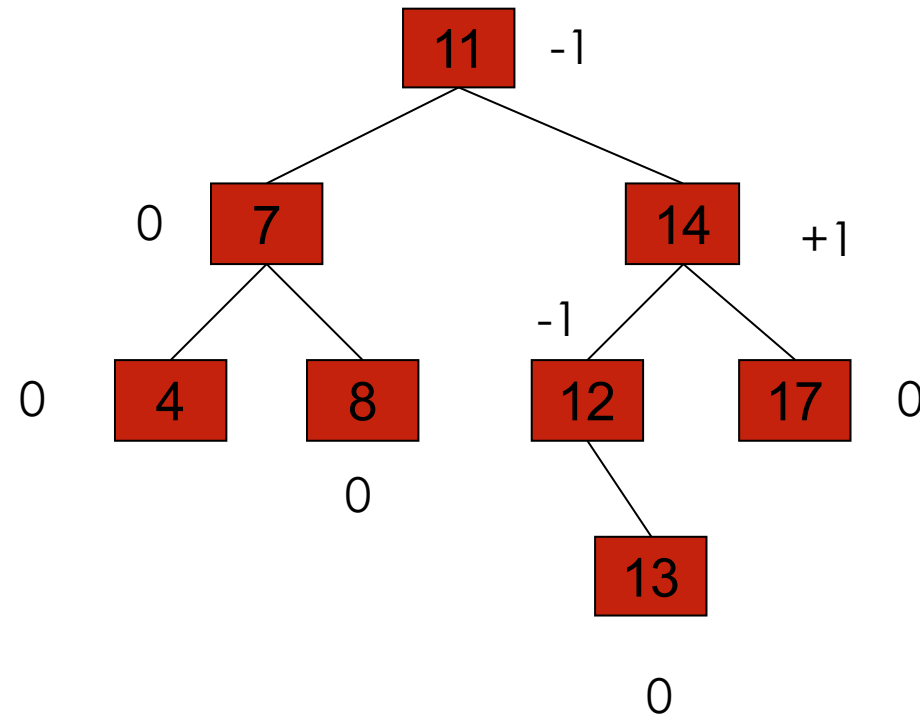
Node 12 is unbalanced.

Rule: If balance factor is greater than 1, then the current node is unbalanced and we are either in Left Left case or Left Right case. To check whether it is Left Left case or Left Right case, get the balance factor of left subtree. If balance factor of the left subtree is greater than or equal to 0, then it is Left Left case, else Left Right case.

According to the rule, the BF of left subtree (node 11) is 0. So apply LL rotation.

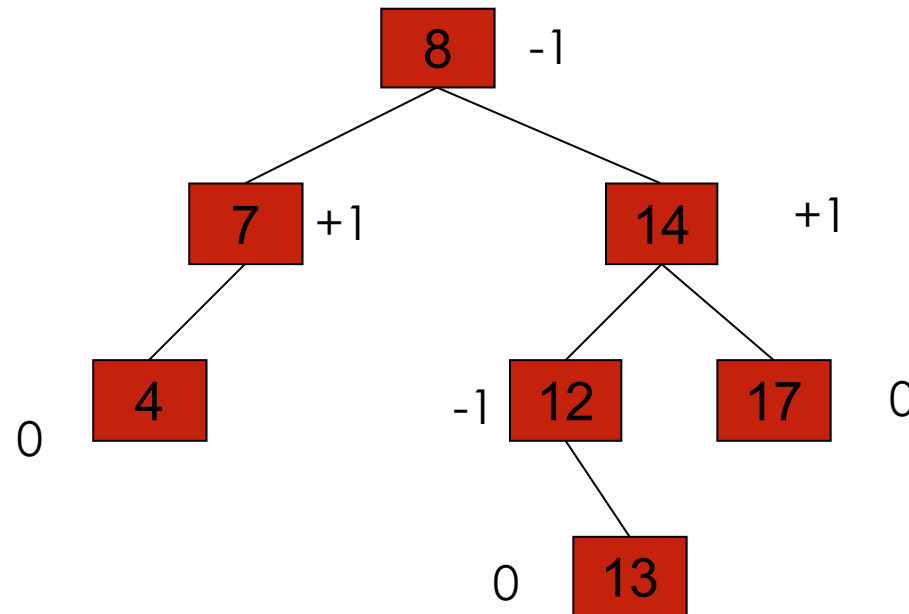
AVL Tree Deletion Example 1:

- **Balanced!** Remove 11



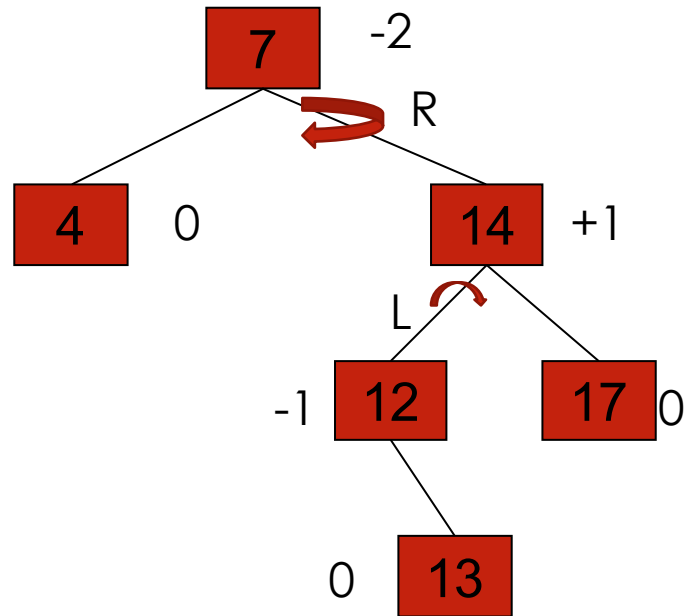
AVL Tree Deletion Example 1 :

- Remove 11, replace it with the largest in its left branch. The resultant tree is balanced



AVL Tree Example:

- Remove 8, replace it with largest in the left branch. Resultant tree is unbalanced



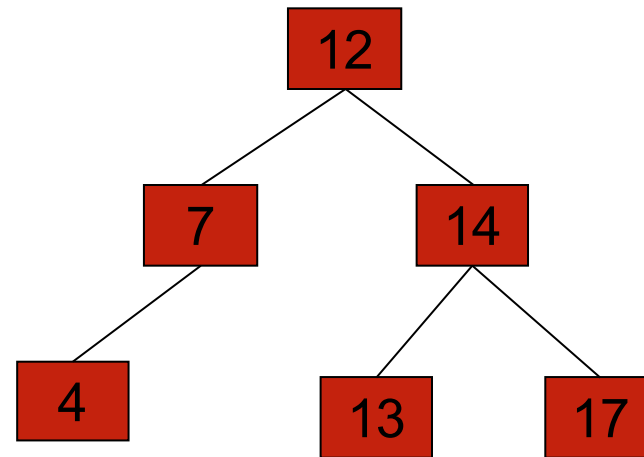
Node 7 is unbalanced.

RULE: If balance factor is less than -1, then the current node is unbalanced and we are either in Right Right case or Right Left case. To check whether it is Right Right case or Right Left case, get the balance factor of right subtree. If the balance factor of the right subtree is smaller than or equal to 0, then it is Right Right case, else Right Left case.

According to the rule, RL case will be applied

AVL Tree Deletion Example 1:

- **Balanced!!**





END ☺

